

圆柱形药材烘干过程的传热传质建模与求解

摘 要

干燥是决定中药材成品品质的关键工序，热风烘干包含预热平衡与恒温干燥两个阶段。本文以长 25 cm、半径 2 cm 的圆柱形药材为对象，建立内部温度场与水分浓度场时空演化的数学模型，在数值方法上采用守恒型有限体积法来离散空间，进而确定满足含水率指标的烘干时长。

针对问题一，因长径比 $L/R = 12.5$ ，且时长仅为半小时，轴向端部效应可忽略。我们首先将药材视为无限长圆柱，建立径向一维非稳态传热-传质模型：温度场由 Fourier 定律描述，水分场由 Fick 第二定律描述，中心轴取零通量条件，表面取 Robin 型对流换热与对流传质条件；烘房时变环境取自附件 1，并经 Savitzky-Golay 保形滤波平滑。然后在空间上以有限体积法离散、时间上以 Crank-Nicolson 格式推进数值求解。结果表明 1800 s 时表面温度 **36.7710 °C**、中心 **33.5711 °C**；表面含水率降至 **1.5102 kg/kg**，距中心 1 cm 以内仍高于 2.53 kg/kg，呈“干壳湿芯”特征。

针对问题二，烘干过程长达 2 ~ 3 天，端部效应此时不可忽略，我们直接对真实圆柱建立模型。由于预热和恒温干燥阶段参数不同，我们先结合滑动均温与升温斜率双条件，得到预热与恒温两阶段分界时刻 $t_c = 6780 \text{ s} = 1.8833 \text{ h}$ ，然后采用后向欧拉格式配合 Picard 迭代与共轭梯度法进行数值求解。结果表明 3 h 时中心温度 **49.8952 °C**、径向温差收缩至 0.06 °C；中心含水率 **1.9237 kg/kg**、表面 1.0791 kg/kg，呈“内湿外干”分布。

针对问题三，因含水率沿径向由外向内递增，中截面中心 $(r, z) = (0, L/2)$ 处最高且下降最慢，故“各处水分浓度均低于 0.15 kg/kg”等价于该点达标。在二维轴对称模型上采用自适应变阶 BDF 算法进行数值求解（与问题四同一算法， $\text{rtol} = 2 \times 10^{-7}$ ，步长由局部误差估计自动选取），并以守恒型有限体积格式保证水分守恒；当全域最大含水率严格低于阈值时，取交叉时刻之后的下一个整秒，得 $t_{\text{dry}} = 56.6517 \text{ h}$ ，此时全域最大含水率为 **0.1499998 kg/kg**（四位小数显示 0.1500）。

针对问题四，考虑水分流失导致的尺寸收缩（详见附件 2：半径由 2.0 cm 收缩至 1.2 cm），我们首先通过坐标变换，以材料（随体）坐标 $\xi = r/R(t)$ 将移动域化为固定域，然后在 (ξ, z) 半域上以节点控制体有限体积离散，水分通量取 Kirchhoff 积分均值（5 点 Gauss 求积）以避免强非线性 D 的假性界面阻力，用隐式 BDF 积分并以事件定位达标时刻，得 $t_{\text{dry}} = 51.0908 \text{ h}$ 。

关键词: 药材烘干 传热传质 有限体积法 移动边界 自适应 BDF

一 问题重述

1.1 问题背景

中药材干燥是决定成品品质的关键工序之一，热风烘干因其设备简单、操作方便而被广泛采用。热风烘干过程一般包含**预热平衡**与**恒温干燥**两个阶段，通过调控烘房内的温度与水分浓度（含湿量）环境完成药材的脱水。若工艺参数选取不当，容易出现干燥效率低、能耗高、成品品质不稳定等问题；而传统的试验优化方法成本高、周期长。因此，需要借助数理分析与数值仿真手段，建立药材烘干过程中温度场与水分浓度场的变化规律模型，为工艺优化提供依据。本文以圆柱形中药材为研究对象，在已知烘房温湿度时变环境与药材物性参数的条件下，求解药材内部温度与水分浓度的时空分布，并进一步确定满足干燥指标所需的烘干时间。

1.2 问题提出

在不改变原题意的前提下，将赛题重述为如下四个子问题：

1. **问题一（预热平衡阶段建模与求解）**：对于长 25 cm、半径 2 cm 的圆柱形药材，烘干开始时温度为 28 °C、水分浓度（干基含水率）为 2.55 kg/kg。在附件 1 给出的烘房温度与水分浓度时变环境下，建立预热平衡阶段药材温度与水分浓度变化规律的数学模型（物性参数见附录 2），并按表 1、表 2 格式给出 100 ~ 1800 s 内若干指定时刻、距药材中心 0 ~ 2 cm 处的温度与水分浓度，完整结果按要求保存至 result1.xlsx，所有结果保留四位小数。
2. **问题二（整个烘干过程建模与求解）**：烘干过程通常持续 2 ~ 3 天，预热平衡与恒温干燥两阶段的工艺参数有所不同。建立整个烘干过程中药材温度与水分浓度变化规律的数学模型（经验公式统一采用附录 3），并按表 3、表 4 格式给出前 3 h 内每隔 0.5 h、距药材中心 0 ~ 2 cm 处的温度与水分浓度，完整结果保存至 result2.xlsx。
3. **问题三（满足含水率指标的烘干时间）**：以药材各处水分浓度均低于 0.15 kg/kg 作为干燥达标的判据，确定药材烘干所需的总时间（单位：h），并按表 5 格式给出每隔 6 h、距药材中心每隔 0.5 cm 处的水分浓度，完整结果保存至 result3.xlsx。
4. **问题四（考虑尺寸收缩的烘干时长）**：实际烘干过程中，药材会因水分流失而发生尺寸变化。在考虑该尺寸变化（附件 2）的前提下，采用附录 4 的经验公式重新确定药材的烘干时长，并按表 6 格式给出每隔 6 h、距药材中心每隔 0.5 cm 处的水分浓度，完整结果保存至 result4.xlsx。

二 问题分析

本文研究圆柱形中药材在热风烘干过程中温度场与水分浓度场的时空演化规律。四个子问题由浅入深、层层递进：问题一建立预热平衡阶段的基础传热—传质模型；问题二将其推广至持续 2 ~ 3 天的完整烘干过程，并引入随温湿度变化的物性参数；问题三在问题二模型的基础上确定满足含水率指标的烘干时间；问题四进一步考虑水分流失引起的尺寸收缩，将固定域问题推广为移动边界问题。

2.1 问题一的分析

问题一属于非稳态传热—传质问题，在数学上归结为圆柱域上一组抛物型偏微分方程的初边值问题。其难点有二：药材长径比很大 ($L/R = 12.5$)，直接三维瞬态求解代价过高，需作几何降维；烘房温湿度为时变边界条件，且实测数据含测量噪声，需先平滑处理。

求解思路：因 $L \gg R$ ，忽略轴向端部效应与周向不均匀性，将圆柱药材退化为无限长圆柱，问题降维为一维径向问题，仅研究 $r \in [0, R]$ 。温度场由 Fourier 定律描述，水分场由 Fick 第二定律描述；中心轴由轴对称性给出零通量条件，表面为 Robin 型对流边界，环境时变量 $T_a(t)$ 、 $C_a(t)$ 取自附件 1 并经 Savitzky–Golay 平滑。

2.2 问题二的分析

问题二将研究范围由 30 分钟的预热阶段扩展到持续 2 ~ 3 天的完整烘干过程。相较问题一，本问的本质变化可归结为四点。

其一，**控制方程不再解耦：**附录 3 中 ρ, c_p, k 均随含水率 C 变化， D 还含 Arrhenius 温度项， ρc_p 不能提到时间导数之外，传热与传质双向耦合。

其二，**几何不能再降维：**问题一降维的依据是 30 min 的预热尺度远小于轴向扩散的特征时间；恒温干燥持续数天，端面交换经轴向扩散向内渗透，端部效应不再可忽略，且物性随 C, T 变化后也失去了严格的轴对称依据。

其三，**阶段分界时刻需识别：**两阶段物性公式不同，而 t_c 题目并未给出，须依据附件 1 的实测温度判定；单一温度阈值易被测温噪声提前触发，需要更稳健的判据。

其四，**环境需外推、时间跨度大：**附件 1 仅覆盖前 4 h，其后须以恒温段平台均值外推；全过程长达数天，需兼顾数值稳定性与计算效率。

求解思路：先利用数据驱动求解出阶段分界时刻 t_c ，然后在三维柱坐标有限体积分框架下联立求解两方程，两阶段物性按识别出的 t_c 分段取用并保持切换连续。

2.3 问题三的分析

问题三在问题二模型的基础上引入干燥达标判据：药材各处水分浓度均低于 $C_{\text{crit}} = 0.15 \text{ kg/kg}$ 。关键在于如何判断全局水分浓度低于所给阈值

求解思路：在问题二模型上作长时间推进，由于水分浓度沿径向由外向内递增，中截面中心 $(r, z) = (0, L/2)$ 处最高、下降最慢，故“各处均达标”可以转化为“中截面中心 $C(0, L/2, t) < 0.15$ ”，多维达标判断归结为对单点时间序列的监控，问题转化为临界时间的确定。

2.4 问题四的分析

问题四考虑实际烘干中因水分流失导致的尺寸收缩，即求解域外边界 $R(t)$ 随时刻移动，构成移动边界问题。难点有三：其一，半径收缩使空间离散域逐时刻变化，固定网格难以直接套用；其二，表 6 要求输出到“药材表面”的水分浓度，而该位置本身随时刻移动；其三， $R(t)$ 全程不小于 1.2 cm，故“距中心 1.5 cm”及更远的位置在收缩后已不存在，输出时须留空而不能外推。

求解思路：先采用材料（随体）坐标变换 $\xi = r/R(t)$ 将移动区间 $[0, R(t)]$ 映射为固定区间 $[0, 1]$ ，把收缩视为材料自身的运动：均匀径向收缩的材料速度为 $v_r = (\dot{R}/R)r$ 。 $R(t)$ 的连续表达式由附件 2 数据作保形分段三次插值得到。然后在考虑材料自身运动的情况下改写传热方程和传质方程。最后采用有限体积格式进行空间离散和隐式 BDF 方法积分进行数值计算。

基于上述分析，问题一至问题四的完整求解路线如图 1 所示：四者共用同一套数据预处理与阶段分界识别，随后按各自的几何维度与物理假设建立模型、选取数值格式，最终输出对应结果表并统一经过数值验证。

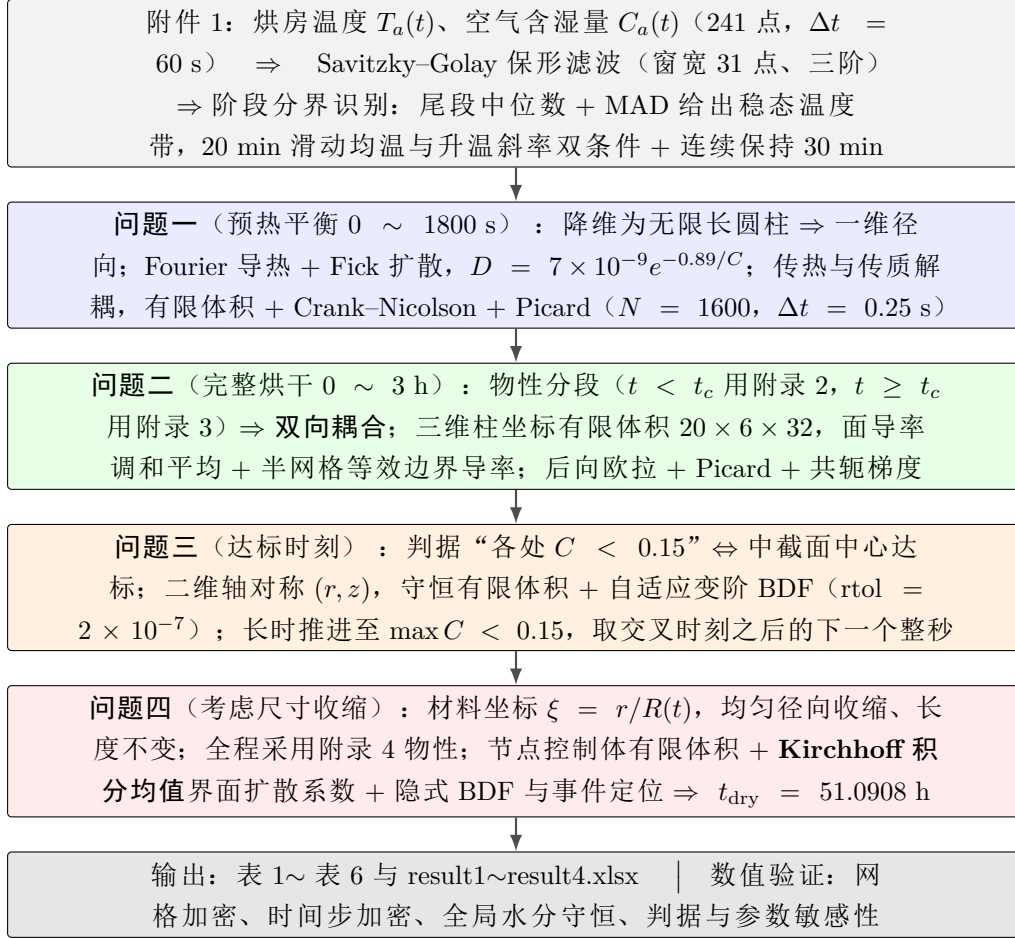


图 1: 本文求解流程图 (问题一至问题四)

三 模型假设

结合药材物性与热风烘干的工艺特点, 本文建模作如下假设:

1. **连续介质与各向同性假设:** 药材视为均匀、连续、各向同性的多孔介质, 内部不存在生成或消耗热量与水分的内源项; 同一时刻的物性仅随含水率 (及温度) 变化, 不随空间位置变化。因此控制方程中不含源项, 物性可作为状态量的函数统一写入。
2. **导热与扩散定律假设:** 药材内部的热传递服从 Fourier 定律, 水分迁移服从 Fick 第二定律, 水分以浓度梯度驱动的扩散为主, 忽略毛细力、压力梯度与重力引起的迁移, 也不考虑水分以气态形式在孔隙内的独立流动。
3. **对流边界假设:** 药材表面与烘房空气之间以对流方式换热与传质, 对流换热系数 h 与对流传质系数 h_m 取附录给定的常数, 不随温度、含湿量、风速及表面含水率变化。

4. **环境均匀且单向作用假设：**烘房内温度与空气含湿量处处均匀，其随时间的变化规律由附件 1 完全给定，且不因药材的吸放热与失水而改变。即环境构成一组给定的时变边界条件，药材的存在不影响烘房环境。
5. **降维假设：**预热平衡阶段（问题一）持续时间短，轴向端部效应与周向不均匀性可忽略，药材退化为无限长圆柱，仅保留径向变化；恒温干燥持续数天，端面交换经轴向扩散向内渗透而不可忽略，故问题二保留完整三维模型，问题三利用几何与受载的轴对称性采用二维轴对称模型。
6. **尺寸假设：**问题一至问题三中，药材在干燥过程中尺寸保持不变，即 $R = 2\text{ cm}$ 、 $L = 25\text{ cm}$ 为常数；因水分流失引起的尺寸收缩仅在问题四中予以考虑，直接采用附件 2 给出的半径随时间变化数据 $R(t)$ ，并假定药材作**均匀径向收缩、轴向长度不变**——附件 2 只给出半径的变化，未给出长度变化，故取 $L = 25\text{ cm}$ 保持不变；不能仅凭半径数据推断轴向是否等比例收缩。均匀径向收缩的材料速度为 $v_r = (\dot{R}/R)r$ ，据此可采用材料（随体）坐标 $\xi = r/R(t)$ 。
7. **初始均匀假设：**烘干开始时，药材内部的温度与水分浓度处处均匀，分别为 $T_0 = 28\text{ }^\circ\text{C}$ 与 $C_0 = 2.55\text{ kg/kg}$ 。
8. **分段物性假设：**预热平衡阶段的物性取附录 2 给出的常数，恒温干燥阶段的物性取附录 3 给出的随含水率变化的关系式；两阶段以识别出的临界时刻 t_c 为界切换，切换处只更换物性参数而不重置状态量，保证温度场与水分浓度场连续。

四 符号说明

本文用到的物理量符号及其含义、单位见表 1，数值方法与求解参数的记号见表 2。除特别说明外，各符号在四个问题中含义一致。

符号	含义	单位
时空变量与几何量		
t	时间	s (部分结果用 h)
r	径向坐标, 到药材中心的距离	m (结果表用 cm)
φ	周向坐标 (问题二三维模型)	rad
z	轴向坐标, 原点由各问计算域给出	m
ξ	归一化径向坐标 $\xi = r/R(t)$ (问题四)	—
$R, R(t), R_0$	药材半径; 问题四为随时间收缩的半径, $R_0 = R(0) = 2$ cm	m
L, H	药材轴向长度与半轴向长度 $H = L/2$ (问题四计算域)	m
Ω, Γ	计算域及其外边界 (问题二、三)	—
dA	边界面面积微元	m ²
v_r, v_z	材料径向、轴向收缩速度 $v_r = (\dot{R}/R)r, v_z \equiv 0$ (问题四)	m/s
状态量与环境量		
$T(r, t)$	药材内部温度 (问题一至三)	°C (附录公式用 K)
$C(r, t)$	药材水分浓度 (干基含水率, 问题一至三)	kg/kg
$U(\xi, z, t)$	材料坐标下的温度 $U = T(R(t)\xi, z, t)$ (问题四)	°C
$M(\xi, z, t)$	材料坐标下的水分浓度 $M = C(R(t)\xi, z, t)$ (问题四)	kg/kg
$T_a(t), C_a(t)$	烘房环境温度、空气含湿量 (上划线表示尾段平台均值)	°C、kg/kg
T_Γ, C_Γ	外边界 Γ 处的温度、水分浓度 (问题二); 问题三的表面值记为 C_{surface}	°C、kg/kg
T^*, σ	尾段温度中位数、MAD 换算的稳健波动尺度 (问题二判据)	°C
$\tau, \bar{T}, \bar{t}, \beta(\tau)$	判据的候选时刻、窗口均温、时间均值与最小二乘升温斜率 (问题二判据)	s、°C、°C/min
T_0, C_0	药材初始温度、初始水分浓度	°C、kg/kg
$C_{\text{crit}}, C_{\text{max}}$	干燥达标含水率、全域最大水分浓度 (达标判据)	kg/kg
$t_c, t_{\text{dry}}, t_{\text{end}}$	阶段分界时刻、烘干总时长、时间推进终止时刻	s、h
J	水分通量: J_{ij} 为节点间通量密度, $J_{\text{surface}}, J_{\text{out}}$ 为边界总通量	kg/(m ² ·s)、kg/s
物性参数		
ρ	药材密度	kg/m ³
c_p	药材比热容	J/(kg·K)
k	热传导系数	W/(m·K)
h	对流换热系数	W/(m ² ·K)
h_m	对流传质系数	m/s
$D, \hat{D}_{ij}, \Phi$	水分扩散系数; 相邻节点间的积分平均系数与 Kirchhoff 势函数 $\Phi'(C) = D(C, T)$ (问题四)	m ² /s
α	热扩散系数 $\alpha = k/(\rho c_p)$	m ² /s
Bi_m	传质毕渥数 $Bi_m = h_m R/D$	—

表 1: 主要符号说明 (物理量)

符号	含义	单位
离散参数与求解记号		
N, N_t	问题一径向网格区间数、时间步总数	—
n_r, n_z	问题三、四径向、轴向网格区间数（问题四取半轴向）	—
i, j, n, m	网格节点、时间步与迭代步指标（ n 亦表示外法向与窗口样本数）	—
$\tilde{T}_i, C_{ij}$	控制体中心处的温度、节点处的水分浓度	$^{\circ}\text{C}$ 、 kg/kg
T_h, C_h	温度、水分浓度的网格离散向量（问题三求解）	—
Z	联合未知量 $Z = (T_h, C_h)^T$ ；牛顿修正量 δZ 、迭代量 $Z^{(m)}$ （问题三求解）	—
p, a_j, β_p	BDF 阶数与 BDF 系数（问题三求解）	—
$\mathcal{F}, G, \mathcal{J}, I$	半离散方程右端函数、牛顿迭代残差、雅可比矩阵与单位矩阵（问题三求解）	—
$K_{\max}$	Picard 迭代次数上限（问题二求解）	—
$\text{rtol}, \text{atol}, \hat{e}$	求解器相对、绝对容差与局部截断误差估计（问题三求解）	—
$g(t)$	达标事件函数 $g(t) = \max_{i,j} C_{ij}(t) - 0.15$ （问题四求解）	kg/kg
$V_{ij}, \Delta \ell_{ij}$	控制体体积、相邻节点中心距（问题四求解）	m^3 、 m
$\Delta r, \Delta z$	径向、轴向空间步长	m
Δt	时间步长（ ΔT 、 ΔC 为迭代变化量）	s

表 2: 数值方法与求解参数记号

五 模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 问题一模型的建立

1. 数据预处理

模型求解所需的烘房环境条件由附件 1 给出：共 241 个采样点，时间范围 $t \in [0, 14400]$ s、采样间隔 60 s，记录了预热平衡阶段烘房温度 $T_a(t)$ 与空气含湿量 $C_a(t)$ 的时变数据。实测平台段温度存在测量噪声（ $t \geq 10800$ s 段标准差约 0.153°C ），由于实验测量数据存在一定的随机波动，若直接将各测量点连接，温度曲线会出现较明显的锯齿状波动。为减小测量噪声对后续计算的影响，同时尽可能保留原始数据的变化趋势，采用 Savitzky–Golay 滤波对实验数据进行平滑处理。滤波窗口取 31 个数据点，即 31 min，多项式阶数取 3。

平滑处理后，温度平台段的标准差降至 0.027°C ，较平滑前降低约 82%，说明数据波动得到明显抑制。同时，平滑前后平台段的平均温度偏差仅为 $3.2 \times 10^{-4}^{\circ}\text{C}$ ，对整体温度水平的影响可以忽略。对于升温阶段，前 30 min 内平滑前后的最大温度差为 0.17°C ，仅占该阶段升温幅度 13.5°C 的约 1.3%，因此平滑处理不会明显

改变原始升温趋势。在此基础上，对平滑后的环境温度 $T_a(t)$ 和环境含水率 $C_a(t)$ 进行线性插值，得到连续时间下的环境温度和含水率，并将其作为问题一传热传质模型的边界条件。

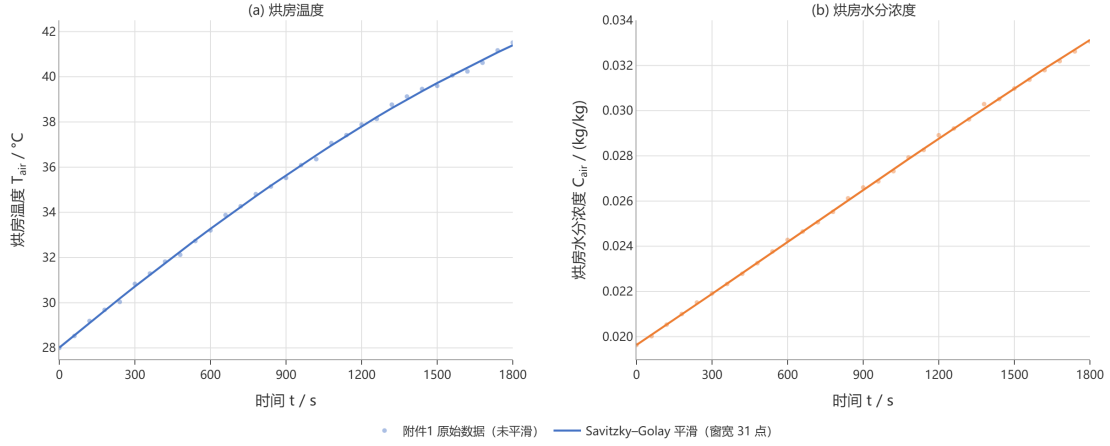


图 2: 前 30 min (问题一计算区间) 烘房温度与空气含湿量的变化

2. 问题描述与几何简化

预热平衡阶段，某圆柱形中药材（长 $L = 25$ cm、半径 $R = 2$ cm）置于烘房内，初始温度 $T_0 = 28$ °C、初始水分浓度（干基含水率） $C_0 = 2.55$ kg/kg。烘房温度 $T_a(t)$ 与空气含湿量 $C_a(t)$ 随时间的非稳态变化由附件 1 给出。由于 $L \gg R$ ，忽略轴向端部效应，将药材视为无限长圆柱，问题退化为一维径向问题，仅研究 $r \in [0, R]$ 方向的温度场与水分浓度场。

3. 控制方程

考虑药材内部的传热与传质过程。由于药材可近似视为圆柱体，且其长径比较大，因此忽略轴向温度梯度和水分梯度，并假设温度场与水分迁移过程均具有轴对称性。设 $T(r, t)$ 为时刻 t 、径向位置 r 处的温度， $C(r, t)$ 为时刻 t 、径向位置 r 处的水分浓度。

对于药材内部的热传递过程，根据 Fourier 热传导定律，并参考圆柱形物料干燥过程中的热质耦合模型 [1]，在上述假设下，可得到圆柱坐标系下的径向一维非稳态热传导方程：

$$\rho c_p \frac{\partial T}{\partial t} = \frac{k}{r} \frac{\partial}{\partial r} \left(r \frac{\partial T}{\partial r} \right), \quad 0 < r < R, \quad t > 0 \quad (1)$$

其中， ρ 为药材密度， c_p 为比热容， k 为导热系数。对于药材内部的水分迁移过程，根据 Fick 第二定律及扩散理论 [3]，并参考圆柱形物料干燥过程中的传质模型 [1]，在考虑有效扩散系数随水分浓度变化的情况下，建立水分传递方程。根据

圆柱坐标系下梯度与散度算子的坐标变换关系 [2]，并结合轴对称条件，可得到径向一维水分传递方程：

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C) \frac{\partial C}{\partial r} \right), \quad 0 < r < R, \quad t > 0 \quad (2)$$

$$D(C) = 7 \times 10^{-9} e^{-0.89/C} \quad (\text{m}^2/\text{s}). \quad (3)$$

4. 初始条件与边界条件

初始时刻药材内部温度、水分浓度均匀：

$$T(r, 0) = T_0, \quad C(r, 0) = C_0. \quad (4)$$

中心 $r = 0$ 处由对称性得：

$$\left. \frac{\partial T}{\partial r} \right|_{r=0} = 0, \quad \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0. \quad (5)$$

表面 $r = R$ 处为对流换热与对流传质边界：

$$-k \left. \frac{\partial T}{\partial r} \right|_{r=R} = h [T(R, t) - T_a(t)], \quad (6)$$

$$-D \left. \frac{\partial C}{\partial r} \right|_{r=R} = h_m [C(R, t) - C_a(t)]. \quad (7)$$

5. 模型参数

由附录 2，相关物性参数取值见表 3。

参数	符号	数值
密度	ρ	820 kg/m ³
比热容	c_p	2600 J/(kg · K)
热传导系数	k	0.36 W/(m · K)
对流换热系数	h	25 W/(m ² · K)
对流传质系数	h_m	8×10^{-7} m/s
初始温度	T_0	28 °C
初始水分浓度	C_0	2.55 kg/kg

表 3: 问题一相关参数（附录 2）

需要指出，本问中 ρ, c_p, k, h, h_m 均为常数，扩散系数 D 仅依赖 C 而与 T 无关，故传热方程 (1) 与传质方程 (2) 相互解耦，可分别独立求解。

5.1.2 问题一模型的求解

1. 数值方法

采用**有限体积法**（守恒形式）对方程 (1)、(2) 进行径向空间离散：将柱坐标散度项写成通量守恒形式，控制体界面取几何坐标 $r_{i\pm 1/2}$ ，得到三对角线性方程组。时间方向采用 **Crank–Nicolson** 格式（对线性扩散方程，Crank–Nicolson 格式具有二阶时间精度，并具有良好的稳定性；对于当前含非线性扩散系数的耦合问题，其整体稳定性还需要结合具体离散方程和迭代过程分析 [4]。）；由于 $D(C)$ 非线性，传质方程每步采用 **Picard 迭代**（滞后系数）求解。环境温湿度 $T_a(t), C_a(t)$ 由附件 1 数据经 Savitzky–Golay 平滑后线性插值得到。径向取 $N = 1600$ 个区间（节点 $r_i = i\Delta r$, $i = 0, \dots, N$, $\Delta r = 1.25 \times 10^{-5}$ m），时间步长 $\Delta t = 0.25$ s。经网格与时间步加密验证，结果变化小于 10^{-4} ，满足四位小数精度要求。求解流程见算法 1。

Algorithm 1 问题一瞬态传热–传质数值求解

```

1: 初始化:  $T \leftarrow T_0$ ,  $C \leftarrow C_0$ , 径向网格  $r_i = i\Delta r$  ( $i = 0, \dots, N$ )
2: for  $n = 1$  to  $N_t$  do
3:    $t_n \leftarrow n\Delta t$ ; 由附件 1 插值得到环境  $T_a(t_n), C_a(t_n)$ 
4:   用 Crank–Nicolson 求解温度场  $T^n$ （解三对角方程）
5:   for  $k = 1$  to 3 do ▷ Picard 迭代
6:     由  $C^{n,k}$  计算  $D(C)$ , 组装并求解水分场  $C^{n,k+1}$ 
7:   end for
8: end for
9: return 各时刻、各径向位置的温度  $T$  与水分浓度  $C$ 

```

2. 结果

按表 1、表 2 格式，给出 100, 300, 600, 900, 1200, 1500, 1800 s 时刻、距中心 0, 0.5, 1, 1.5, 2 cm 处的温度与水分浓度，见表 4、5。1800 s 内每隔 1 s、每 0.1 cm 的完整结果已保存至 result1.xlsx。各径向位置的温度与水分浓度随时间的变化曲线见图 3。

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	28.0001	28.0004	28.0045	28.0347	28.1795
300	28.0406	28.0630	28.1511	28.3716	28.8438
600	28.4510	28.5338	28.8035	29.3218	30.1837
900	29.3243	29.4574	29.8724	30.6132	31.7453
1200	30.5454	30.7133	31.2269	32.1143	33.4157
1500	31.9952	32.1844	32.7574	33.7284	35.1150
1800	33.5711	33.7696	34.3669	35.3667	36.7710

表 4: 预热平衡阶段药材温度（单位：°C）

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2471
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

表 5: 预热平衡阶段药材水分浓度（单位：kg/kg）

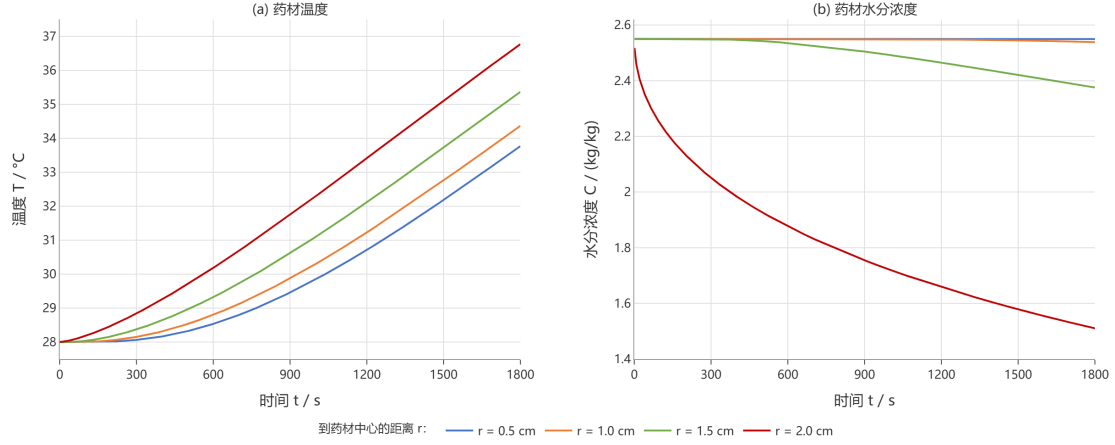


图 3: 预热平衡阶段药材温度（左）与水分浓度（右）随时间的变化

3. 结果分析

由表 4 可知，药材表面升温较快、中心明显滞后：1800 s 时表面已达 36.77°C ，而中心仅 33.57°C ，径向温差约 3.2°C 。这符合圆柱体热传导的滞后特性——热扩散系数 $\alpha = k/(\rho c_p) \approx 1.69 \times 10^{-7} \text{ m}^2/\text{s}$ ，特征时间 $R^2/\alpha \approx 2370 \text{ s}$ 大于 1800 s，故中心尚未充分热透。

由表 5 可知，表面水分浓度由 2.55 kg/kg 降至 1.51 kg/kg ，降幅沿径向由外向内迅速衰减： $r = 1.5 \text{ cm}$ 处降至 2.38 kg/kg ，而 $r \leq 1 \text{ cm}$ 的内部基本保持 2.55 kg/kg 不变，呈现明显的“干壳湿芯”特征。其原因是：传质毕渥数 $Bi_m = h_m R/D(C_0) \approx 3.2 > 1$ ，外部传质阻力小于内部扩散阻力，故表面水分浓度能较快跟随环境含湿量下降；而扩散系数 $D(C_0) = 7 \times 10^{-9} e^{-0.89/C_0} \approx 4.9 \times 10^{-9} \text{ m}^2/\text{s}$ 仍然很小，30 min 内水分扩散的特征深度 $\sqrt{Dt} \approx 2.9 \text{ mm}$ ，与表中表面向内约 1 cm 的范围内才出现明显失水在量级上一致，更深处的内部尚未受到影响。

5.2 问题二模型的建立与求解

5.2.1 问题二模型的建立

1. 分段变物性参数的非线性耦合传热传质模型

与问题一仅涉及预热平衡阶段不同，问题二覆盖持续 2 ~ 3 天的完整烘干过程，预热平衡与恒温干燥两阶段的物性参数并不相同。按题目要求，经验公式统一采用附录 2 与附录 3，两阶段物性分段取值见表 6；扩散系数则分段取为

$$D = \begin{cases} 7 \times 10^{-9} e^{-0.89/C}, & t < t_c, \\ 2.4 \times 10^{-3} e^{-0.45/C} e^{-3850/T}, & t \geq t_c, \end{cases} \quad (\text{m}^2/\text{s}) \quad (8)$$

其中 t_c 为两阶段的临界分界时刻, 附录 3 各式中的 T 取热力学温度 (单位 K)。

阶段	时间区间	ρ (kg/m ³)	c_p (J/(kg · K))	k (W/(m · K))
预热平衡	$t < t_c$	820	2600	0.36
恒温干燥	$t \geq t_c$	$650 + 128C$	$1450 + 2736\frac{C}{C+1}$	$0.21 + 0.38\frac{C}{C+1}$

表 6: 问题二分段物性参数 (预热平衡阶段取自附录 2, 恒温干燥阶段取自附录 3)

需要强调的是, 恒温干燥阶段的 ρ 、 c_p 、 k 均随水分浓度 C 变化, D 同时依赖 C 与温度 T (含 Arrhenius 型温度项)。因此, 物性参数会随着温度场和水分场的变化而动态变化, 使传热与传质过程相互影响。已有研究针对多孔湿物料干燥过程建立了温度场与含水率场的耦合偏微分方程模型, 并通过数值方法求解温度和含水率的时空变化 [5]。因此, 本模型同样需要将传热方程与传质方程联立求解, 这是本问与问题一在模型结构上的本质区别。

2. 计算域与三维控制方程

问题一将药材退化为无限长圆柱, 其依据是 30 min 的预热时间尺度远小于轴向扩散的特征时间。但恒温干燥阶段持续数天, 轴向端面的对流交换将经轴向扩散逐步向内部渗透, 端部效应不再可以忽略; 同时物性随 C, T 变化, 方程本身也不再有严格的轴对称简化依据。为严格保留各控制体的真实通量与边界面积, 本问不再作降维处理, 直接在柱坐标 (r, φ, z) 下建立完整三维模型 [1]。

计算域为整个圆柱体

$$\Omega = \{(r, \varphi, z) : 0 \leq r \leq R, 0 \leq \varphi < 2\pi, -\frac{L}{2} \leq z \leq \frac{L}{2}\}, \quad (9)$$

其中半径 $R = 0.02$ m、长度 $L = 0.25$ m, 与问题一保持一致。

由能量守恒与质量守恒, 控制方程写成守恒 (散度) 形式

$$\rho c_p \frac{\partial T}{\partial t} = \nabla \cdot (k \nabla T), \quad \frac{\partial C}{\partial t} = \nabla \cdot (D \nabla C). \quad (10)$$

利用柱坐标下的梯度与散度算子 [2] 展开, 得径向、周向、轴向三个方向均保留的控制方程

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right) + \frac{1}{r^2} \frac{\partial}{\partial \varphi} \left(k \frac{\partial T}{\partial \varphi} \right) + \frac{\partial}{\partial z} \left(k \frac{\partial T}{\partial z} \right), \quad (11)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right) + \frac{1}{r^2} \frac{\partial}{\partial \varphi} \left(D \frac{\partial C}{\partial \varphi} \right) + \frac{\partial}{\partial z} \left(D \frac{\partial C}{\partial z} \right). \quad (12)$$

3. 初始条件与边界条件

初始时刻药材内部温度与水分浓度均匀：

$$T(r, \varphi, z, 0) = T_0 = 28\text{ }^{\circ}\text{C}, \quad C(r, \varphi, z, 0) = C_0 = 2.55\text{ kg/kg}. \quad (13)$$

药材侧面 $r = R$ 与两个端面 $z = \pm L/2$ 均直接暴露于烘房环境，采用对流换热与对流传质条件：

$$-k \left. \frac{\partial T}{\partial n} \right|_{\Gamma} = h [T_{\Gamma} - T_a(t)], \quad -D \left. \frac{\partial C}{\partial n} \right|_{\Gamma} = h_m [C_{\Gamma} - C_a(t)], \quad (14)$$

其中 Γ 表示外边界， $\partial/\partial n$ 为外法向导数， $h = 25\text{ W}/(\text{m}^2 \cdot \text{K})$ ， $h_m = 8 \times 10^{-7}\text{ m/s}$ 。周向采用周期性边界；轴线 $r = 0$ 处控制体自然退化，不存在额外的边界面，故无需附加条件。环境温度与空气含湿量 $T_a(t), C_a(t)$ 由附件 1 按时间线性插值给出，附件 1 给定范围之外按问题一中由附件 1 确定的恒温段平台值外推。

4. 阶段分界时刻 t_c 的稳态统计判定

分段物性的切换点 t_c 必须依据实测数据客观确定。若采用单一温度阈值（例如烘房温度达到平台值的 95% 或 99.5%），容易被测温噪声提前触发，也无法保证此后烘房确实保持恒温。为此，本文同时检查“接近平台”与“升温速率足够小”两个条件，并附加持续性要求，步骤如下。

第 1 步（估计平台）：用附件 1 中 $t \geq 10800\text{ s}$ 的尾段数据估计恒温平台。需要强调，本步的尾段统计量一律取自原始实测数据—— σ 应反映真实的测量噪声；若改用平滑序列的 MAD，量到的只是滤波器自身的平坦度（ σ 会被压小约 7 倍、稳态带随之收紧，临界时刻将被人为推迟）。即：**对流边界条件用平滑序列，阶段判定用原始序列**。为降低异常波动的影响，平台中心取中位数，噪声尺度取 MAD（中位数绝对偏差）换算的稳健标准差：

$$T^* = \text{median}(T_a), \quad \sigma = 1.4826 \cdot \text{median} |T_a - T^*|. \quad (15)$$

第 2 步（滑动统计量）：对每个候选时刻 τ ，用区间 $[\tau - 1200, \tau]$ （即 20 min）内的观测值计算滑动均温 $\bar{T}(\tau)$ ，并用最小二乘估计升温斜率 $\beta(\tau)$ （单位 $^{\circ}\text{C}/\text{min}$ ）：

$$\bar{T}(\tau) = \frac{1}{n} \sum_{j=1}^n T_a(t_j), \quad \beta(\tau) = \frac{\sum_j (t_j - \bar{t}) [T_a(t_j) - \bar{T}(\tau)]}{\sum_j (t_j - \bar{t})^2}. \quad (16)$$

第 3 步（双条件与持续性）：候选点 τ 须同时满足

$$|\bar{T}(\tau) - T^*| \leq 2\sigma, \quad |\beta(\tau)| \leq \frac{3\sigma}{20}, \quad (17)$$

且上述两个条件在 τ 之后连续保持 1800 s。第一个合格的候选点即定义为临界时刻 t_c 。该持续性判断属于离线识别：用后续 30 min 的数据确认 τ 确实进入稳

定段，但将阶段分界回标到 τ ；若用于实时控制，则只能在 $\tau + 1800\text{ s}$ 才能确认，不能提前获知未来数据。

由附件 1 数据计算，各统计量汇总于表 7，识别结果为 $t_c = 6780\text{ s} = 1.8833\text{ h}$ 。

统计量	数值	含义
T^*	50.0030 °C	尾段稳态中位温度
σ	0.2209 °C	MAD 换算的稳健波动尺度
2σ 温度带	$\pm 0.4418\text{ °C}$	滑动均温允许偏差
斜率上限	0.0331 °C/min	20 分钟窗的趋稳约束
临界窗均温	49.5820 °C	$\tau = t_c$ 时的 20 分钟均值
临界窗斜率	0.0184 °C/min	$\tau = t_c$ 时的回归斜率
临界时刻 t_c	6780 s = 1.8833 h	预热结束、恒温干燥开始

表 7: 阶段分界时刻的稳态统计判定结果

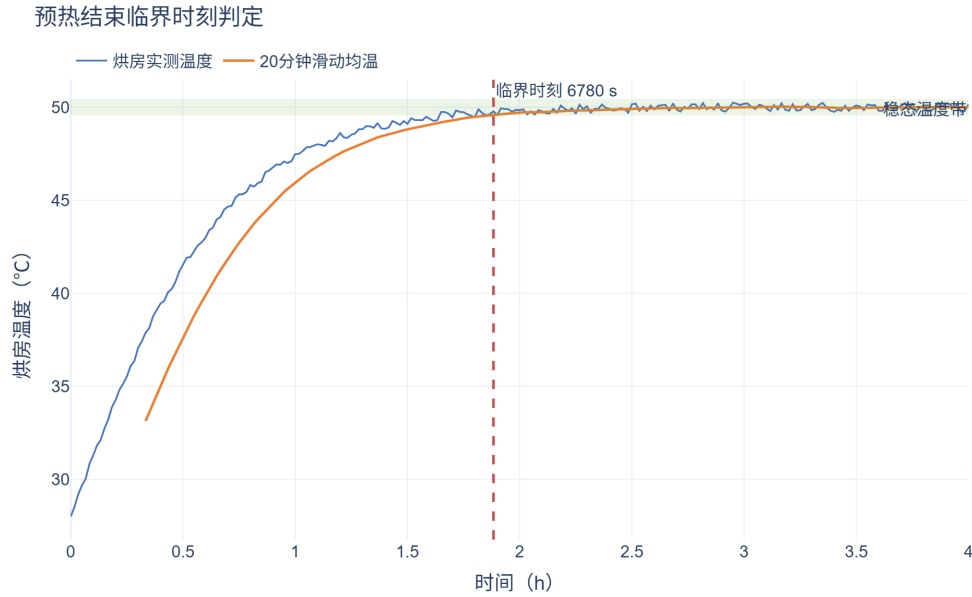


图 4: 预热结束临界时刻判定

绿色带为尾段中位温度与 MAD 给出的稳态温度带，橙线为 20 分钟滑动均温，红虚线为识别出的临界时刻 6780 s。

由图 4 可见，滑动均温在 6780 s 处进入稳态温度带，升温斜率同时低于规定上限，并在随后 30 min 内持续满足两项条件，故将该点识别为预热结束时刻。该判据把“是否进入恒温阶段”建立在平台接近程度、升温斜率与持续时间三项证据之上，比单一进度阈值更能抵抗温度波动。

5. 分段物性的连续切换

临界点处只切换物性参数，不重新赋初值：温度场与水分浓度场在切换处继承前一步的结果，因而状态连续。数值实现上，若某时间步跨越 t_c ，程序先把该步截断到 t_c ，下一步再启用附录 3 的物性公式，从而避免把同一个时间步同时解释为两个阶段。这样既保证了物性切换的物理含义明确，又避免了切换处的数值振荡。

5.2.2 问题二模型的求解

1. 数值方法

采用有限体积法在三维柱坐标控制体上离散方程 (11)、(12)：径向 20、周向 6、轴向 32，共 $20 \times 6 \times 32 = 3840$ 个控制体，控制体体积由 $r\Delta r\Delta\varphi\Delta z$ 精确给出，径向、周向、轴向的面通量均按真实几何面积计算。相邻控制体之间的面通量采用调和平均折算的面导率，以正确反映串联的导热（扩散）阻力；位于边界的控制体保留半网格内部阻力与外对流阻力串联的等效边界导率，而非直接以控制体中心值近似表面值。

时间方向采用后向欧拉（隐式）格式，时间步长 $\Delta t = 5$ s；由于热容 ρc_p 与扩散系数 D 均随 C, T 变化，每一步内采用 Picard 迭代（滞后系数）更新温度、水分浓度与物性，收敛判据取

$$\max\left(\frac{\|\Delta T\|_\infty}{50}, \frac{\|\Delta C\|_\infty}{2.55}\right) < 10^{-8}, \quad (18)$$

即温度与水分浓度的相对变化同时足够小。每一步的线性方程组为对称正定稀疏系统，采用共轭梯度法配合对角预条件求解。求解流程见算法 2。

Algorithm 2 问题二完整烘干过程三维传热-传质数值求解

```
1: 初始化:  $T \leftarrow T_0$ ,  $C \leftarrow C_0$ ,  $t \leftarrow 0$ , 组装三维控制体几何量
2: 由附件 1 按式 (15)–(17) 识别临界时刻  $t_c$ 
3: while  $t < t_{\text{end}}$  do
4:    $t_{\text{new}} \leftarrow \min(t + \Delta t, t_{\text{end}})$ 
5:   if  $t < t_c < t_{\text{new}}$  then
6:      $t_{\text{new}} \leftarrow t_c$  ▷ 时间步截断到阶段分界
7:   end if
8:   由  $t_{\text{new}}$  插值得到环境  $T_a(t), C_a(t)$ 
9:   按  $t < t_c$  与否选择附录 2 或附录 3 的物性公式
10:  for  $m = 1$  to  $K_{\text{max}}$  do ▷ Picard 迭代 (上限  $K_{\text{max}}$ )
11:    以调和平均组装三维有限体积系数矩阵与等效边界导率
12:    后向欧拉求解温度场  $T^{n+1}$ 
13:    由  $T^{n+1}, C^n$  更新  $D$ , 求解水分浓度场  $C^{n+1}$ 
14:    if 式 (18) 成立 then
15:      break
16:    end if
17:  end for
18:   $t \leftarrow t_{\text{new}}$ 
19: end while
20: 提取中截面径向剖面, 输出表 8、9 与 result2.xlsx
```

最终输出圆柱中截面上从轴线到侧面的径向结果, 距离取 $0, 0.1, \dots, 2.0$ cm 共 21 个位置。其中轴线值与侧面值由外推与边界条件给出: 轴线处由径向抛物线外推 $T(0) = [9\tilde{T}_1 - \tilde{T}_2]/8$, 侧面处由对流边界条件 (14) 反解得到; 中间位置由控制体中心值线性插值。

2. 结果

按表 3、表 4 格式, 给出前 3 h 内每隔 0.5 h、距药材中心 0, 0.5, 1, 1.5, 2 cm 处的温度与水分浓度, 分别见表 8 与表 9。前 3 h 内每隔 1 s、距药材中心每隔 0.1 cm 的完整结果已保存至 result2.xlsx。

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	33.5769	33.7772	34.3736	35.3721	36.7750
1.0	42.3313	42.4742	42.8928	43.5714	44.4819
1.5	47.1226	47.1845	47.3647	47.6531	48.0317
2.0	49.0152	49.0364	49.0971	49.1926	49.3187
2.5	49.6166	49.6258	49.6526	49.6960	49.7541
3.0	49.8952	49.8997	49.9126	49.9330	49.9594

表 8: 3 小时内药材的温度（单位：°C）

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	2.5500	2.5496	2.5365	2.3710	1.5106
1.0	2.5459	2.5342	2.4525	2.1234	1.2283
1.5	2.5194	2.4847	2.3324	1.9267	1.0495
2.0	2.4321	2.3706	2.1548	1.7174	1.1859
2.5	2.1695	2.0959	1.8846	1.5633	1.1751
3.0	1.9237	1.8637	1.6923	1.4260	1.0791

表 9: 3 小时内药材的水分浓度（单位：kg/kg）

图 5 与图 6 分别给出中截面温度与水分浓度的时空分布。两图均采用“左热力图、右径向剖面”的双栏形式：左图以颜色表示整个 $0 \sim 3$ h 过程中距轴线不同距离处的数值演化，红色虚线标出临界时刻 $t_c = 1.8833$ h；右图直接给出 0.5, 1, 1.5, 2, 2.5, 3 h 六个典型时刻的径向剖面曲线，便于定量比较同一时刻中心与表面的差异。这种画法保留了三维曲面的全部信息，同时避免了曲面视角遮挡读数的问题。

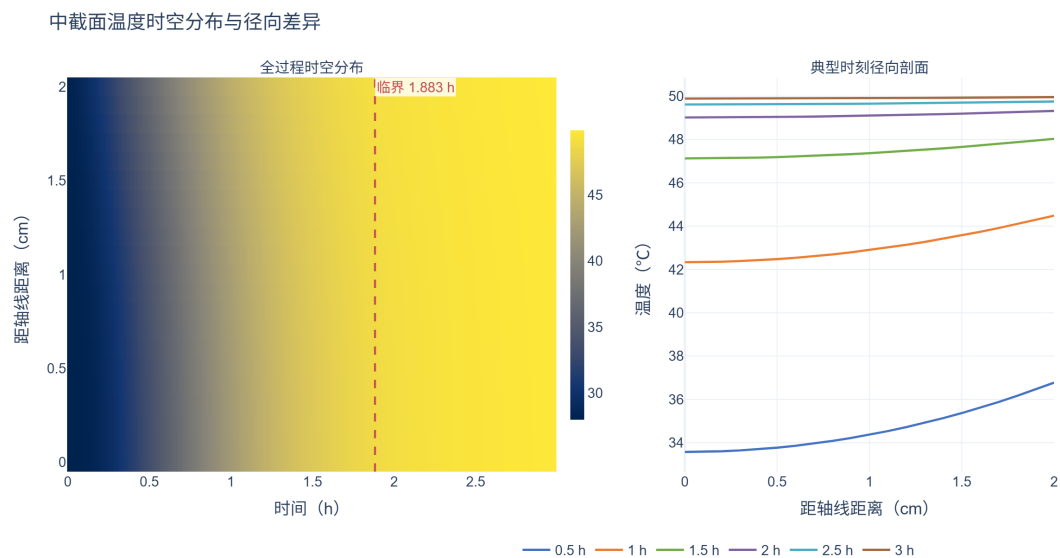


图 5: 中截面温度时空分布与径向差异
(左: 全过程热力图, 红色虚线为 6780 s 的阶段分界; 右: 六个典型时刻的径向剖面)

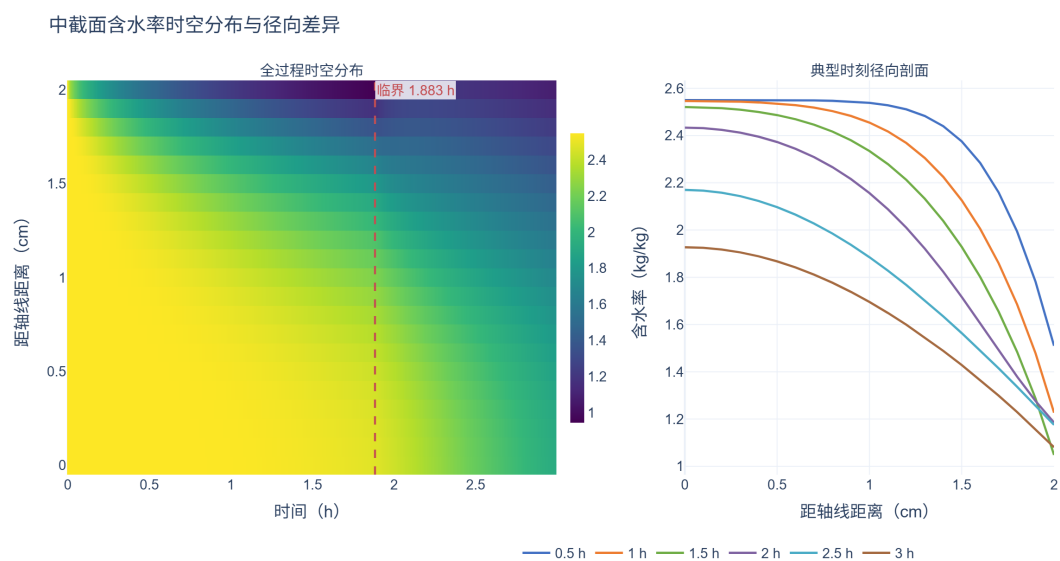


图 6: 中截面含水率时空分布与径向差异
(左: 全过程热力图, 红色虚线为 6780 s 的阶段分界; 右: 六个典型时刻的径向剖面)

3. 结果分析

(1) 阶段分界。临界点 $6780\text{ s} = 1.8833\text{ h}$ 位于 1.5 h 与 2.0 h 之间, 因此前 1.5 h 主要受预热平衡参数控制, 2.0 h 后逐渐体现恒温干燥参数的影响。由表 8 可

见, 1.5 h 至 2.0 h 间中心温度由 47.1226 °C 升至 49.0152 °C, 升温速率明显降低, 表明药材温度已逐渐接近烘房平台温度。

(2) **温度场**。温度由表面向中心逐步传递, 早期径向温差较大, 随后迅速减小。0.5 h 时表面与中心温差为 3.20 °C, 至 3.0 h 仅为 0.06 °C; 此时中心和侧面温度分别为 49.8952 °C 和 49.9594 °C, 均已接近 50 °C。这说明药材在预热后期已基本热透, 温度场趋于均匀, 后续温度对水分迁移的影响主要体现在扩散系数 D 的 Arrhenius 项中。图 5 进一步表明温度梯度主要沿径向分布, 且临界时刻前后温度场连续, 仅物性参数发生切换。

(3) **水分浓度场**。水分浓度呈明显的“内湿外干”特征。3 h 时中心含水率为 1.9237 kg/kg, 侧面仅为 1.0791 kg/kg, 径向差值约为 0.84 kg/kg。这是由于表面失水受对流传质边界直接控制, 而内部水分主要依靠扩散向外迁移, 扩散过程相对较慢。图 6 显示, 含水率随时间整体下降, 且干燥过程由表及里逐步推进; 临界时刻前后色带连续, 说明分段参数切换未造成浓度场跳变。

需要注意的是, 表 9 中 $r = 2$ cm 处的含水率在 1.05 ~ 1.51 kg/kg 间存在小幅波动。这主要与对流边界条件中的传质系数 h_m 、扩散系数 D 及空气含湿量 $C_a(t)$ 随时间变化有关, 而径向内部区域的含水率始终下降, 符合干燥过程的物理规律。

(4) **判据的敏感性与适用性**。本文综合平台接近程度、升温斜率和持续时间判定恒温阶段, 并对关键阈值进行敏感性分析, 详细结果见附录。结果表明, 合理调整判据参数后, 临界时刻变化较小, 说明该判据具有较好的稳定性和抗测温波动能力。因此, 取 $t_c = 6780$ s 作为两阶段分界是合理的, 但具体阈值仍应结合设备控温精度与工艺要求进行调整。

5.3 问题三模型的建立与求解

5.3.1 问题三模型的建立

1. 二维轴对称模型

问题三要求在完整烘干过程基础上确定药材内部所有位置均达到 $C_{\text{crit}} = 0.15$ kg/kg 的最早时刻。圆柱药材关于轴线对称, 且主要传热传质过程由侧面和两个端面控制, 因此采用二维轴对称模型, 计算域为

$$\Omega = \{(r, z) : 0 \leq r \leq R, 0 \leq z \leq L\}, \quad (19)$$

其中

$$R = 0.02 \text{ m}, \quad L = 0.25 \text{ m}. \quad (20)$$

温度场 $T(r, z, t)$ 满足变物性热传导方程

$$\rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(rk(C)\frac{\partial T}{\partial r}\right) + \frac{\partial}{\partial z}\left(k(C)\frac{\partial T}{\partial z}\right), \quad (21)$$

干基水分浓度 $C(r, z, t)$ 满足

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(rD(C, T)\frac{\partial C}{\partial r}\right) + \frac{\partial}{\partial z}\left(D(C, T)\frac{\partial C}{\partial z}\right). \quad (22)$$

依据附录 3, 物性参数取

$$\rho(C) = 650 + 128C, \quad (23)$$

$$c_p(C) = 1450 + 2736\frac{C}{C+1}, \quad (24)$$

$$k(C) = 0.21 + 0.38\frac{C}{C+1}, \quad (25)$$

$$D(C, T) = 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left(-\frac{3850}{T}\right), \quad (26)$$

其中 T 使用开尔文温度。

2. 初始条件与边界条件

初始温度和水份浓度均匀:

$$T(r, z, 0) = 28\text{ }^{\circ}\text{C}, \quad C(r, z, 0) = 2.55\text{ kg/kg}. \quad (27)$$

中心轴采用轴对称条件:

$$\frac{\partial T}{\partial r}(0, z, t) = 0, \quad \frac{\partial C}{\partial r}(0, z, t) = 0. \quad (28)$$

侧面 $r = R$ 和两个端面 $z = 0, L$ 均采用对流换热与对流传质边界:

$$-k(C)\frac{\partial T}{\partial n} = h[T - T_a(t)], \quad (29)$$

$$-D(C, T)\frac{\partial C}{\partial n} = h_m[C - C_a(t)]. \quad (30)$$

其中 $T_a(t)$ 、 $C_a(t)$ 由附件 1 的 Savitzky-Golay 平滑序列按时间插值得到, 附件 1 覆盖范围 (4 h) 之外按尾段平台均值外推; $h = 25\text{ W}/(\text{m}^2 \cdot \text{K})$, $h_m = 8 \times 10^{-7}\text{ m/s}$ 。

3. 物性与阶段说明

与问题二不同，问题三**全程采用附录 3 的经验公式**（式 (23)–(26)），物性不随阶段切换；环境边界同样取附件 1 的平滑实测序列（见下节），因此本问的时间推进不含任何由阶段分界触发的参数跳变。

问题二识别出的临界时刻 $t_c = 6780 \text{ s} = 1.8833 \text{ h}$ 在本问中沿用为 **阶段说明**：预热平衡阶段（ $t < t_c$ ）与恒温干燥阶段（ $t \geq t_c$ ）的起止仍按该时刻叙述，便于与问题二的结果对照，但它不参与本问的物性或边界条件计算。

5.3.2 问题三模型的求解

1. 有限体积离散（守恒格式）

相邻控制体在同一内部面上使用大小相等、方向相反的通量。将全部控制体求和后，内部面通量严格抵消，只保留侧面和两个端面的边界传质通量：

$$J_{\text{surface}} = h_m (C_a - C_{\text{surface}}) dA. \quad (31)$$

其中 C_{surface} 为表面处的水分浓度， dA 为边界面积微元， J_{surface} 为全部边界面上的表面传质总量。该格式从离散层面保证全局水分守恒。

2. 自适应 BDF 时间推进

把计算域按径向 n_r 个区间、轴向 n_z 个区间剖分为 $(n_r + 1)(n_z + 1)$ 个节点，温度场与水分浓度场在网格上离散后分别记为向量 $T_h, C_h \in \mathbb{R}^{(n_r+1)(n_z+1)}$ ，两者组成联合未知量 $Z = (T_h, C_h)^T \in \mathbb{R}^{2(n_r+1)(n_z+1)}$ 。空间离散后，式 (21)、(22) 连同边界条件化为刚性常微分方程组 $\dot{Z} = \mathcal{F}(t, Z)$ ，其中 $\mathcal{F}$ 为半离散方程的右端函数， $\dot{Z}$ 表示 Z 对时间的导数。刚性来源于网格尺度上的扩散时间尺度远小于烘干总时长，时间方向因而采用 **变阶变步长的后向差分公式**（Backward Differentiation Formula, BDF）： p 阶 BDF 用当前层 t_{n+1} 与前 p 个历史层 t_{n+1-j} 的线性组合逼近时间导数，

$$\sum_{j=0}^p a_j Z_{n+1-j} = \Delta t_n \beta_p \mathcal{F}(t_{n+1}, Z_{n+1}), \quad (32)$$

其中 $a_0, \dots, a_p, \beta_p$ 为 BDF 系数， $\Delta t_n = t_{n+1} - t_n$ 为第 n 步的时间步长（下标 n 为时间层序号）， Z_{n+1-j} 为 Z 在第 $n+1-j$ 时间层上的取值。一阶（后向欧拉）与二阶格式分别为

$$\frac{Z_{n+1} - Z_n}{\Delta t_n} = \mathcal{F}(t_{n+1}, Z_{n+1}), \quad \frac{3Z_{n+1} - 4Z_n + Z_{n-1}}{2\Delta t_n} = \mathcal{F}(t_{n+1}, Z_{n+1}). \quad (33)$$

式 (32) 关于 Z_{n+1} 是隐式的，每一时间层用**修正牛顿迭代**求解。记残差函数 G 与雅可比矩阵 $\mathcal{J}$

$$G(Z) = \sum_{j=0}^p a_j Z_{n+1-j} - \Delta t_n \beta_p \mathcal{F}(t_{n+1}, Z), \quad \mathcal{J}(Z) = \frac{\partial \mathcal{F}}{\partial Z}(t_{n+1}, Z), \quad (34)$$

则第 m 次迭代的近似解 $Z^{(m)}$ 由下述**稀疏线性方程组**给出, 其中 I 为单位矩阵、 m 为迭代步序号、 δZ 为牛顿修正量:

$$\left(\frac{a_0}{\Delta t_n} I - \beta_p \mathcal{J} \right) \delta Z = G(Z^{(m)}), \quad Z^{(m+1)} = Z^{(m)} - \delta Z. \quad (35)$$

由于 $\mathcal{F}$ 只把相邻控制体耦合起来, $\mathcal{J}$ 的非零元仅出现在网格邻接位置。 $n_r = 24$ 、 $n_z = 50$ 时未知数为 $2 \times 25 \times 51 = 2550$, 非零元 24892 个; 按该稀疏结构作有限差分并用稀疏 LU 分解求解式 (35)。阶数 $p \in [1, 5]$ 与步长 Δt_n 由局部截断误差估计自动选取, 要求误差估计 $\hat{e}$ 满足 $\|\hat{e}\| \leq \text{rtol} \|Z\| + \text{atol}$, 其中 rtol 、 atol 分别为求解器的相对容差与绝对容差; 本文取 $\text{rtol} = 2.0 \times 10^{-7}$ 、 $\text{atol} = 2.0 \times 10^{-9}$, 并限制 $\Delta t_n \leq 300$ s。BDF 的稳定域覆盖左半平面 ($p = 1, 2$ 为 A -稳定, $p = 3, 4, 5$ 为 $A(\alpha)$ -稳定), 因此步长由精度要求决定, 而不受稳定性限制。

3. 结果

计算得到药材内部所有位置均**严格**满足 $C < 0.15$ kg/kg 的时刻为

$$t_{\text{dry}} = 56.6517 \text{ h} \quad (= 203946 \text{ s}), \quad (36)$$

该时刻全域最大水分浓度为

$$C_{\text{max}} = 0.1499998 \text{ kg/kg}, \quad (37)$$

全精度值为 $0.1499997659 \text{ kg/kg} < 0.15$, 余量 2.4×10^{-7} ; 四位小数输出时显示为 0.1500, 是严格不等式判定与格式舍入共同作用的结果。

表 10 给出每隔 6 h、径向每隔 0.5 cm 的水分浓度, 末行为结束时刻 t_{dry} 所在的时刻。

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
6.0000	1.0170	0.9882	0.9015	0.7550	0.5327
12.0000	0.4563	0.4433	0.4032	0.3298	0.1646
18.0000	0.2988	0.2915	0.2685	0.2247	0.0853
24.0000	0.2374	0.2323	0.2161	0.1842	0.0667
30.0000	0.2052	0.2011	0.1882	0.1625	0.0599
36.0000	0.1851	0.1817	0.1708	0.1487	0.0566
42.0000	0.1712	0.1682	0.1586	0.1390	0.0548
48.0000	0.1610	0.1583	0.1496	0.1318	0.0537
54.0000	0.1530	0.1505	0.1426	0.1261	0.0529
56.6517	0.1500	0.1476	0.1399	0.1240	0.0527

表 10: 药材烘干过程中中截面 ($z = L/2$) 的水分浓度 (单位: kg/kg)

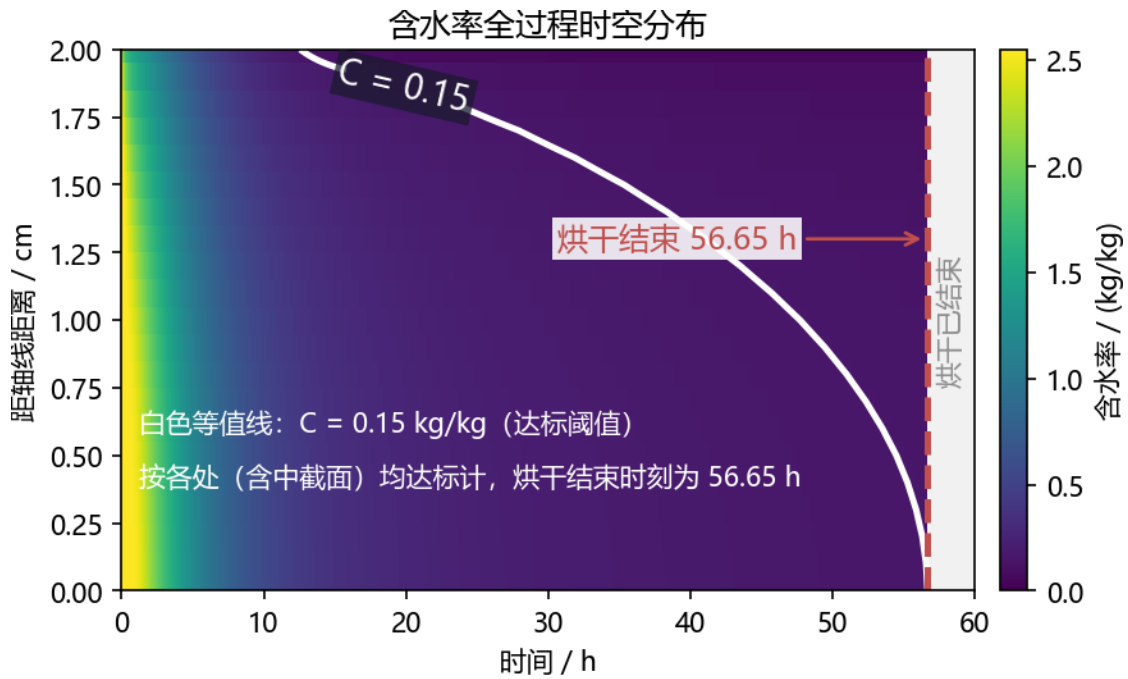


图 7: 第三问药材含水率时空分布与径向差异 (达标阈值 0.15 kg/kg)

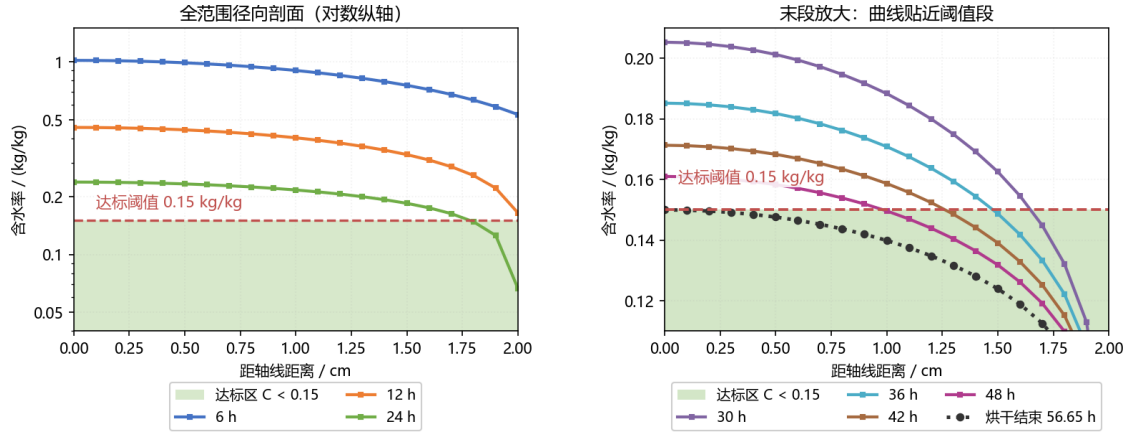


图 8: 第三问药材含水率时空分布与径向差异 (达标阈值 0.15 kg/kg)

由表 10 可见, 水分浓度始终满足中心高、表面低的空间分布, 说明干燥过程由外向内推进。6 h 时中心水分浓度仍约为 1.0170 kg/kg, 54 h 时已降至 0.1530 kg/kg, 至 56.6517 h 全域达标。

表中给出的是中截面 $z = L/2$ 上的径向剖面 (与问题一、二、四同一口径)。数值上可以核实: 结束时刻全域最大含水率恰出现在该剖面中心 $r = 0$ 处, 故表中最后一行 ($t = 56.6517$ h) 的径向最大值即全域最大值, 该行数值可直接反映达标判据。最后一行的 0.1500 系四位小数舍入显示, 全精度值 0.1499997659 仍严格小于 0.15 kg/kg。

5.4 问题四模型的建立与求解

5.4.1 问题四模型的建立

1. 几何假设与移动边界

实际烘干中, 药材因失水而发生尺寸变化。附件 2 只给出半径随时间的变化 (145 个采样点、间隔 1800 s、覆盖 0 ~ 72 h、全程单调不减), 未给出长度的变化, 故假设药材作均匀径向收缩、轴向长度保持 $L = 25$ cm; 不能仅凭半径数据推断轴向是否等比例收缩。 $R(t)$ 由附件 2 数据作保形分段三次 Hermite 插值 (PCHIP) 得到, 以避免普通高阶拟合出现振荡或非物理的半径反弹。

初始半径 $R_0 = 2$ cm, 6 h 即降至 1.374 cm, 24 h 后基本稳定在 1.2 cm——收缩高度集中在前 12 h, 而全过程长达 51 h, 因此绝大部分干燥是在已经收缩后的域内进行的。利用几何与受载关于中截面的对称性, 取中截面为 $z = 0$, 只计算半域

$$0 \leq r \leq R(t), \quad 0 \leq z \leq H = 0.125 \text{ m}. \quad (38)$$

2. 材料坐标下的控制方程

控制方程的关键在于：干基含水率 C 是“每单位干物质所含水的质量”，它随固体骨架一起运动，而不是“水质量/当前体积”的体积浓度。均匀径向收缩的材料速度为

$$v_r(r, t) = \frac{\dot{R}(t)}{R(t)} r, \quad v_z = 0. \quad (39)$$

记随材料运动的物质导数 $D_t = \partial_t + v_r \partial_r$ 。令

$$\xi = \frac{r}{R(t)}, \quad U(\xi, z, t) = T(R(t)\xi, z, t), \quad M(\xi, z, t) = C(R(t)\xi, z, t), \quad (40)$$

链式法则给出

$$\partial_t C|_r = M_t - \frac{\dot{R}}{R} \xi M_\xi, \quad v_r C_r = \frac{\dot{R}}{R} \xi M_\xi, \quad (41)$$

两式恰好相消。因此采用相对固相的有效扩散模型时，固定材料坐标上的控制方程为

$$\rho(M)c_p(M)U_t = \frac{1}{R^2\xi} \partial_\xi(\xi k(M)U_\xi) + \partial_z(k(M)U_z), \quad (42)$$

$$M_t = \frac{1}{R^2\xi} \partial_\xi(\xi D(M, U)M_\xi) + \partial_z(D(M, U)M_z). \quad (43)$$

3. 物性参数

题目在本问明确给出附录 4 的经验公式，故主方案从初始时刻起全程采用附录 4，不移植问题三中附录 2→附录 3 的阶段切换：

$$\rho = 760 + 90M, \quad c_p = 1850 + 2150 \frac{M}{M+1}, \quad k = 0.12 + 0.20 \frac{M}{M+1}, \quad (44)$$

$$D = 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{M}\right) \exp\left(-\frac{3850}{U+273.15}\right). \quad (45)$$

其中 U 以 $^\circ\text{C}$ 存储、仅在指数式中换算为 K ； $M/(M+1)$ 是整体分式。第 5.8 节另给出沿用问题三预热切换方式的敏感性结果。

4. 初始条件与边界条件

初始时刻药材内部温度与水分浓度均匀：

$$U(\xi, z, 0) = 28^\circ\text{C}, \quad M(\xi, z, 0) = 2.55 \text{ kg/kg}. \quad (46)$$

中心轴 $\xi = 0$ 与中截面 $z = 0$ 均满足零法向梯度：

$$U_\xi(0, z, t) = M_\xi(0, z, t) = 0, \quad U_z(\xi, 0, t) = M_z(\xi, 0, t) = 0. \quad (47)$$

侧面 $\xi = 1$ 与端面 $z = H$ 均直接暴露于烘房，取 Robin 型对流换热与对流传质条件：

$$-\frac{k}{R}U_\xi = h(U - T_a), \quad -\frac{D}{R}M_\xi = h_m(M - C_a), \quad (48)$$

$$-kU_z = h(U - T_a), \quad -DM_z = h_m(M - C_a), \quad (49)$$

其中 $h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$, $h_m = 8 \times 10^{-7} \text{ m/s}$ 。端面与侧面的交界节点同时计入两类外表面通量；中截面不是暴露端面，故不加对流项。本问继续采用问题三忽略辐射与显式蒸发潜热的简化层级。

环境条件取自附件 1：先作 **Savitzky–Golay 保形滤波**（窗宽 31 点即 31 min、三阶多项式，与问题一、二同一套处理），平台段温度标准差由 0.153 降至 0.027 °C；再在相邻采样点之间线性插值。 $t > 14400 \text{ s}$ 后，取平滑序列在 $t \geq 10800 \text{ s}$ 段的算术平均作为恒温平台

$$T_a = 49.9986 \text{ °C}, \quad C_a = 0.049992 \text{ kg/kg}, \quad (50)$$

该外推是后续数十小时预测所必需的环境假设。

5.4.2 问题四模型的求解

1. 空间离散

采用节点控制体有限体积格式：径向 n_r 个区间、半轴向 n_z 个区间，共 $(n_r + 1)(n_z + 1)$ 个节点，每节点含温度与含水率两个未知量。轴线、侧面、中截面、端面均为节点；以相邻节点中点为控制体界面，中心轴与外边界取相应的半控制体。省略公因子 2π ，控制体体积为

$$V_{ij}(t) = \frac{R(t)^2}{2} (\xi_{i+1/2}^2 - \xi_{i-1/2}^2) \Delta z_j. \quad (51)$$

所有内界面共用同一数值通量、对相邻控制体以相反符号累加，故内部通量求和精确相消。温度导率取相邻节点值的**调和平均**。

水分的界面扩散系数则**不能**简单取调和平均或算术平均。附录 4 的 D 对含水率呈指数型强非线性（近表面 C 由 2.55 降至 0.05 量级），若用两个节点 D 的调和平均，会把粗网格表面的低扩散系数错误地扩展到整个网格间距，产生明显的**假性传质阻力**。为此水分通量取沿相邻节点状态的**积分均值**（即 Kirchhoff 变量差分）

$$\hat{D}_{ij} = \int_0^1 D((1-s)C_i + sC_j, (1-s)T_i + sT_j) ds, \quad J_{ij} = -\hat{D}_{ij} \frac{C_j - C_i}{\Delta \ell_{ij}}, \quad (52)$$

积分用 5 点 Gauss–Legendre 求积公式。在恒温情形，若 $\Phi'(C) = D(C, T)$ ，则该通量对应 $J_{ij} = -[\Phi(C_j) - \Phi(C_i)]/\Delta \ell_{ij}$ ，即 Kirchhoff 变换后的通量差分；当两节点浓度相等时，积分表达式自然退化到该点的扩散系数，避免了直接差商在相近浓度下的消减误差。该处理不对结果作任何人为校正，仍是对原扩散方程的通量离散。

主计算采用径向 96 个区间、半轴向 80 个区间（ 97×81 个节点）。需要注意，在归一化坐标 $\xi = r/R(t)$ 下网格固定，而物理网格 $\Delta r = R(t)/n_r$ 随 R 由 2 cm 缩到 1.2 cm 而变细约 40%，即最粗的物理分辨率出现在初始阶段。

2. 时间积分与事件定位

空间离散后得到刚性常微分方程组，采用隐式 **BDF** 方法积分，并提供与二维邻接结构对应的稀疏 Jacobian 模式。主计算相对容差 2×10^{-7} 、绝对容差 2×10^{-9} ；考虑前 4 h 的边界数据每 60 s 变化，该段最大内部步长取 60 s，其后取 300 s。

烘干时长由**事件定位**给出。设

$$g(t) = \max_{i,j} C_{ij}(t) - 0.15, \quad (53)$$

求其首次由正变负的零点即临界时刻。为满足题目“各处**低于** 0.15 kg/kg”的严格不等式，取下一整秒作为结果表的结束时刻，并在该时刻验证全场最大值确实小于 0.15。内部积分步长与结果表的采样间隔无关：利用求解器的连续输出在 60, 120, 180, ... 秒处求值，而不是把粗时步的结果简单取整。全过程到 72 h 仍不达标时程序报错说明半径数据不足，不擅自生成达标时长。

3. 计算结果

计算得药材内部所有位置均满足 $C < 0.15$ kg/kg 的时刻为

$$t_{\text{dry}} = 51.0908 \text{ h} \quad (183927 \text{ s}), \quad (54)$$

此时药材半径为 1.2000 cm，全场最大含水率为 0.1499999 kg/kg < 0.15 ；临界时刻前 1 s 的全场最大含水率为 0.1500005 kg/kg，事件前后方向正确、量级相符。表 11 给出表 6 所需的径向剖面。

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	药材表面
6.0000	1.7188	1.5370	1.0221	—	0.4205
12.0000	0.7374	0.6544	0.4082	—	0.1669
18.0000	0.4087	0.3686	0.2397	—	0.0892
24.0000	0.2851	0.2610	0.1790	—	0.0673
30.0000	0.2263	0.2094	0.1493	—	0.0595
36.0000	0.1928	0.1796	0.1317	—	0.0560
42.0000	0.1712	0.1604	0.1200	—	0.0541
48.0000	0.1561	0.1468	0.1116	—	0.0530
51.0908	0.1500	0.1413	0.1081	—	0.0526

表 11: 考虑收缩时药材烘干过程的水分浓度（单位：kg/kg，中截面口径）

按题目要求，表 6 与 result4.xlsx 报告**中截面**（ $z = L/2$ ）的径向剖面：题目未指定轴向位置，而“距药材中心某距离处的水分浓度”更接近一个具体位置的局部

值；轴向平均值会被干燥较快的两端拉低，使药材中心显得比实际更干。但烘干时长始终按完整二维场的最大值判定，不能用剖面值是否低于 0.15 来判断达标。为便于核对移动边界，表中另列出该时刻的半径。

由 $R(6\text{ h}) = 1.374\text{ cm}$ 可知，表 6 从第一条记录起 1.5 cm 就已落在药材外部，该位置以“—”标出：既不能填 0，也不能沿固定的 2 cm 圆柱外推，更不能误填环境含湿量。result4.xlsx 按 60 s 采样并附结束时刻，按 0.1 cm 输出实际距离剖面，超出当前半径的位置留空，另列该时刻的真实表面含水率。

图 9 给出中截面水分浓度的时空分布与药材表面的收缩过程。

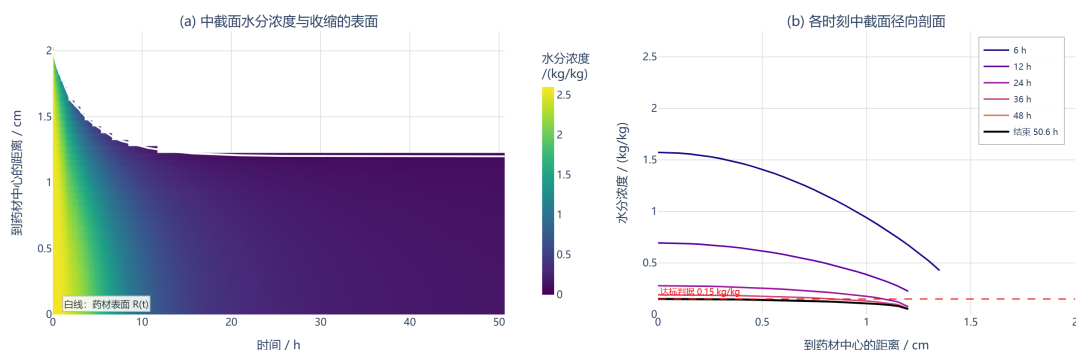


图 9: 问题四中截面水分浓度的时空分布

(左：热力图，白线为随时间收缩的药材表面，灰色区域为收缩后已不存在的部分；右：若干典型时刻的径向剖面，▼ 为该时刻的药材表面)

4. 结果分析

(1) 烘干时长与前三问的比较。本问结果为 51.0908 h，比问题三的 56.6517 h 短约 9.8%。但这一差异不能全部归因于收缩：本问同时更换了扩散系数与热物性公式（附录 4 的 D 在相同条件下比附录 3 小 2~5 倍），两者的作用方向相反。要分离纯收缩效应，需要保持全部物性与环境一致再做对照计算。直观上，半径收缩到原来的 59.9% 使扩散特征时间 R^2/D 缩短到约 0.36 倍，这是收缩使干燥加快的主要机制。

(2) 表面的快速失水与内部的缓慢干燥。6 h 时药材表面已降至 0.4205 kg/kg，而中心仍有 1.7188 kg/kg；表面在 12 h 后即降至 0.15 以下（0.1669 → 平台约 0.0526，已接近烘房空气含湿量 $C_a = 0.049992$ ）。这说明全过程的控速环节始终是内部水分向表面的扩散。该判断与传质毕渥数一致：以 $M \approx 0.2$ 、 $U = 50\text{ }^\circ\text{C}$ 估计 $D \approx 6.2 \times 10^{-10}\text{ m}^2/\text{s}$ ， $Bi_m = h_m R/D \approx 15 \gg 1$ ，即外部传质阻力远小于内部扩散阻力，表面迅速与环境趋于平衡，而中心只能靠缓慢的扩散逐步降低。

(3) 中截面中心即全场最湿点。51.0908 h 时，药材中截面中心的含水率为 0.1499999 kg/kg，恰为整个二维场的最大值；而同一时刻按轴向平均的中心值为

0.1464 kg/kg。若改用轴向平均值判断达标，会得到**偏早**的结束时刻。这也从反面说明：输出表可以用中截面口径，但达标判据必须用完整二维场的最大值。

5. 数值验证

问题四的结束判据是**全场最大含水率**这一点值，而 D 对含水率呈指数型强非线性，因此烘干时长对空间离散较敏感，必须单独做收敛检查——问题三在固定尺寸域上的收敛结论**不能替代**本问。

(1) **空间网格收敛性**。固定时间容差，逐级加密网格，结果见表 12。

径向 $\times$ 半轴向区间数	临界时长/h	相邻级差/min	误差缩减比
24 $\times$ 20	51.11538250	—	—
48 $\times$ 40	51.09577973	-1.176	—
72 $\times$ 60	51.09206574	-0.223	5.27
96 $\times$ 80	51.09076319	-0.078	2.86

表 12: 问题四空间网格收敛性 (Δt 容差固定，轴向区间数指半长度方向)

级差单调递减，末期误差缩减比约 2.9；按 Richardson 外推，96 $\times$ 80 网格的残余误差约为 2.5 s。因此主文件采用 96 $\times$ 80 的结果，并在论文中报告“约 51.1 h”——四位小数只是题目要求的输出格式，并不代表物理预测具有四位小数的精度。

(2) **时间离散的不敏感性**。在 48 $\times$ 40 网格上，将相对容差由 2×10^{-7} 收紧到 2×10^{-8} ，并把后期最大内部步长由 300 s 降到 150 s，临界时刻仅变化 0.000353 s，说明该网格下时间离散误差远小于空间离散误差。

(3) **守恒与平衡检查**。离散格式满足 $\sum_{ij} V_{ij} \dot{C}_{ij} + F_{\text{out}} = 0$ 。由于几何在收缩，这不等价于 $d(\sum VC)/dt = -F_{\text{out}}$ ；干基含水率的干物质加权总量应按参考控制体权重计算（均匀径向收缩时，干物质体积密度与当前控制体体积之积为常数）。程序检验的是内通量相消与参考干物质权重下的离散平衡，不把 $\rho(C)V$ 当作恒定干物质质量。此外，均匀场等于环境且无边界差值时，即使 $R(t)$ 变化，含水率也应保持常数——该检查同样通过；最终瞬时内外通量平衡的相对残差为 0.0（机器舍入精度）。

(4) **环境平滑与物性阶段处理的敏感性**。在 72 $\times$ 60 网格上，若环境输入不做 Savitzky-Golay 平滑而直接用附件 1 原值，临界时长为 51.0915 h，与平滑方案的 51.0921 h 相差约 2 s，影响可忽略。若沿用问题三的阶段处理方式（在 4980 ~ 6780 s 内用 smoothstep 权重在附录 2 与附录 4 之间逐项混合 ρ, c_p, k, D ），在 48 $\times$ 40 网格上得到 50.6860 h，与同网格全程附录 4 的 51.0958 h 相差 0.4097 h（约 24.6 min）——物性阶段假设造成的差异大于已检验的时间数值误差，故本文明确选择题目本问给定的全程附录 4 方案。

(5) **结论与限制。**综合上述检验，本问在给定假设与附录 4 物性下给出的烘干时长为 $t_{\text{dry}} \approx 51.1 \text{ h}$ 。数值收敛验证的是给定假设下离散解的稳定性，不能代替实测干燥曲线对模型的验证；本节所有结论均建立在“均匀径向收缩、长度不变、全程附录 4、忽略蒸发潜热”这组明确假设之上。

六 模型优缺点分析

6.1 优点

- **模型复杂度与问题尺度相匹配：**问题一依据 $L/R = 12.5$ 作一维降维、问题二因长时程端部效应保留完整三维、问题三利用轴对称性回到二维，既避免了“一律三维”造成的计算浪费（问题二全量 3 h 仿真仅约 9 s），也避免了过度简化导致失真。
- **离散格式守恒、边界处理精细：**采用守恒型有限体积法，控制体体积与面通量均按真实几何量计算；内部面导率用调和平均折算，边界面用“半网格内部阻力与外对流阻力串联”的等效导率，而非以控制体中心值近似表面值，从而避免了后者带来的一阶误差。
- **阶段分界判据客观、抗噪且可复现：**以尾段中位数与 MAD 稳健标准差给出稳态温度带，再结合滑动均温与升温斜率双条件及持续性要求判定 t_c ，比单一进度阈值更能抵抗测温噪声；敏感性检验表明三个互不相关的松紧变体给出完全相同的 t_c 。
- **数值方案稳定且精度可控：**时间方向全部采用隐式格式（Crank–Nicolson、后向欧拉、自适应变阶 BDF），对刚性扩散问题无条件稳定，适合 2 ~ 3 天的长时间推进；问题三、四的长时间推进统一为误差可控的自适应 BDF（ $\text{rtol} = 2 \times 10^{-7}$ ），步长由精度而非稳定性决定，计算量较固定步长格式下降约两个数量级。
- **结果的不确定度已量化：**通过空间网格加密、时间步缩小、 t_c 判据阈值扰动三组算例给出偏差量级，并将全域偏差定位到边界反解层，使结论的可靠范围明确可辨。

6.2 缺点与改进方向

- **缺少实测内部数据用于标定与验证：**附件仅给出烘房环境条件，没有药材内部温度与含水率的实测值，模型只能在机理与数值层面作自洽检验。改进方

向：增设少量内部测点，用其反演 h_m 、 D 等难以准确给定的参数，并对模型做实测校验。

- **物性参数做了常数化处理：**问题一至问题三中 h 、 h_m 取常数，未考虑风速与表面含水率的影响；预热阶段亦忽略了温度对扩散系数的 Arrhenius 效应。改进方向：改用 $h_m = f(u)$ 、 $D = D(C, T)$ 的一般形式。
- **热质耦合仍不完备：**模型未计入表面水分汽化带走的热量（相变潜热），也未考虑干燥收缩引起的孔隙结构变化对扩散系数的影响，这在干燥后期含水率很低时可能带来偏差。改进方向：在表面能量平衡中加入汽化潜热项，并把孔隙率变化引入有效扩散系数。
- **结果的位数表述需谨慎：**收敛性检验表明表列数值自第 3 位小数起已不具收敛意义，故正文的物理解释均建立在 10^{-2} 量级的差异上，未对更低位小数作实质性推断。
- **判据阈值仍属建模选择：** 2σ 温度带、20 min 窗长与 30 min 持续性要求缺少设备控温精度与工艺容差的支撑，实际应用前应结合具体烘房条件作标定与灵敏度复核。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、代码调试和图片绘制 (AI 生成绘图代码)，详细使用情况见支撑材料。

参考文献

- [1] SILVA W.P.da, SILVA C.M.D.P.S e, GAMA F.J.A. Estimation of thermo-physical properties of products with cylindrical shape during drying: the coupling between mass and heat[J]. Journal of Food Engineering, 2014, 141: 65-73. DOI: 10.1016/j.jfoodeng.2014.05.010.
- [2] COMSOL. Modeling with PDEs: Coordinate Transformations[EB/OL]. COMSOL Learning Center, 2026[2026-09-10].
<https://www.comsol.com/support/learning-center/article/modeling-with-pdes-coordinate-transformations-47951>
- [3] CRANK J. The Mathematics of Diffusion[M]. 2nd ed. Oxford: Clarendon Press, 1975.

- [4] CRANK J, NICOLSON P. A practical method for numerical evaluation of solutions of partial differential equations of the heat-conduction type[J]. Mathematical Proceedings of the Cambridge Philosophical Society, 1947, 43(1): 50-67. DOI:10.1017/S0305004100023197.
- [5] Cai W, Zhu L, Dong S, Xie G, Li J. Effect of Thermophysical Properties on Coupled Heat and Mass Transfer in Porous Material during Forced Convective Drying. *Advances in Mechanical Engineering*, 2014, 6: 830387. DOI: 10.1155/2014/830387.

A 附录

A.1 收敛性与敏感性检验

为检验数值结果的可靠性，本文在问题二模型上设计了三组算例：固定时间步长 $\Delta t = 5$ s 加密空间网格、固定网格 $20 \times 6 \times 32$ 缩小时间步长，以及扰动阶段分界判据 t_c 的阈值参数。前两组取各自最细配置为参考解，偏差同时给出两种口径：限定在论文表 8、9 实际列出的位置（6 个时刻 $\times$ 5 个径向位置），以及全域（3 h 内每一秒 $\times$ 21 个径向位置）。结果见表 13。

收敛方式	配置	控制体数	$\Delta t/\text{s}$	3 h 中心值		表列位置最大偏差	
				$T/^\circ\text{C}$	C	$\max \Delta T $	$\max \Delta C $
空间加密	$20 \times 6 \times 32$	3840	5	49.8867	1.9237	1.6×10^{-3}	3.6×10^{-3}
	$32 \times 8 \times 48$	12288	5	49.8868	1.9235	4.1×10^{-4}	1.2×10^{-3}
	$48 \times 12 \times 72^\dagger$	41472	5	49.8868	1.9238	—	—
时间加密	$\Delta t = 5$ s	3840	5	49.8867	1.9237	4.5×10^{-3}	3.5×10^{-4}
	$\Delta t = 2.5$ s	3840	2.5	49.8868	1.9237	1.5×10^{-3}	1.2×10^{-4}
	$\Delta t = 1.25$ s †	3840	1.25	49.8868	1.9236	—	—

C 的单位为 kg/kg，偏差单位与所在列自变量一致； † 为该组的参考解。

表 13: 离散参数收敛性检验（问题二：0 ~ 3 h）

由表 13 可见：温度对**时间步长**更敏感—— Δt 由 5 s 减至 1.25 s 使表列偏差由 4.5×10^{-3} 降至 1.5×10^{-3} $^\circ\text{C}$ ，而网格加密只使温度偏差由 1.6×10^{-3} 降至 4.1×10^{-4} $^\circ\text{C}$ ；含水率则对**空间网格**更敏感——网格加密使表列偏差由 3.6×10^{-3} 降至 1.2×10^{-3} kg/kg，而时间步加密仅由 3.5×10^{-4} 降至 1.2×10^{-4} kg/kg。这与物理图像一致：温度场瞬态响应快，精度主要受时间离散制约；含水率在近表面处梯度陡，精度主要受空间离散制约。

全域偏差大于表列偏差，其原因已定位到**边界反解层**： $r = 2$ cm 处的数值并非控制体中心值，而是由对流边界条件 (14) 用半网格阻力**一阶反解**得到，其精度依赖 Δr ；且首个时间步（ $t < 60$ s）表层与环境尚未建立平衡、梯度最陡，反解误差最大（ $t = 5$ s 处不同网格相差达 9.5×10^{-2} kg/kg）。剔除 $t < 60$ s 与 $r = 2$ cm 后，全域含水率偏差降至 4.9×10^{-4} kg/kg 以内，说明该偏差是边界重构的局部效应，不向内部传播；论文表 9 所取的 $t \geq 0.5$ h 各时刻不受其影响。

阶段分界判据 t_c 的阈值敏感性见表 14。

t_c 判据变体	t_c/s	$\Delta t_c/s$	3 h 中心值		表列位置
			$T/^\circ\text{C}$	C	$\max \Delta C $
基准 2σ / 窗 20 min / 持续 30 min	6780	0	49.8867	1.9237	—
偏松 1.5σ	7080	+300	49.8898	1.9417	8.9×10^{-2}
偏严 3σ	6360	-420	49.8831	1.9009	4.9×10^{-2}
斜率上限 $\times 0.5$	7020	+240	49.8891	1.9380	6.2×10^{-2}
窗长 15 min	6540	-240	49.8846	1.9104	3.3×10^{-2}
窗长 30 min	7080	+300	49.8898	1.9417	8.9×10^{-2}
斜率上限 $\times 2$ 、持续 15/60 min	6780	0	49.8867	1.9237	0

偏差相对基准解， C 的单位为 kg/kg；末行列出三个互不相关的松紧变体，其识别结果与基准完全一致。

表 14: t_c 判据阈值的敏感性（网格 $20 \times 6 \times 32$ ， $\Delta t = 5$ s）

由表 14 可见，判据阈值的合理扰动使 t_c 在 6360 ~ 7080 s 之间变动（相对基准 $-6.2\% \sim +4.4\%$ ），而 3 h 中心含水率仅在 1.9009 ~ 1.9417 kg/kg 之间变化（相对极差约 2%），中心温度变化不超过 0.007 $^\circ\text{C}$ 。其中**持续性要求最不敏感**：由 30 min 收紧到 15 min 或放宽到 60 min，识别出的 t_c 完全不变，说明平台段确实已经稳定；而 2σ 温度带与 20 min 窗长的松紧会系统性地提前或推迟 t_c ，两者对结果的影响方向相反、量级相当。

综合来看，模型对离散参数与判据阈值均不敏感：表列数值在离散加密下变化不超过 5×10^{-3} ，在判据阈值扰动下不超过 9×10^{-2} kg/kg。需要说明的是，论文表 8、9 按要求保留四位小数，但上述检验表明第 3 位小数起已不具有收敛意义，可靠的是前两位；因此正文的一切物理解释均建立在 10^{-2} 量级的差异上，未对第 3、4 位小数作实质性推断。

A.2 问题三的数值验证

问题三的达标时刻由全域最大含水率的下穿事件给出，属点值判据，且扩散系数 D 对含水率呈指数型强非线性，因此结束时刻对空间离散较为敏感，需单独检验。围绕该求解过程共做六项检验，结果汇总于表 15。

检验项目	配置与关键结果
空间网格收敛 [†]	生产求解器（自适应 BDF），粗/中/细三级网格的结束时刻分别为 55.489444/56.134167/56.476389 h，单调收敛，观测阶 1.562，Richardson 外推 56.863549 h
时间步长收敛 [‡]	固定步长隐式 RK4， $\Delta t = 2.5/1.25/0.625$ s 的 L_2 误差为 $1.124 \times 10^{-9}/1.146 \times 10^{-10}/0$ ，观测阶 3.294
全局水分守恒	全部控制体的水分变化与累计表面传质通量闭合，归一化相对误差 3.4644×10^{-5}
均匀稳态	均匀场 $T = 50$ °C、 $C = 0.05$ kg/kg 保持 24 h：右端项初值为 0，温度与含水率漂移恒为 0
表面含水率回升	粗/中/细三级网格的最大回升量为 $-0.003855/-0.003862/-0.003864$ （负值即持续下降），均出现于 6660 s
求解器对照	自适应 BDF 与固定步长 $\Delta t = 2.5$ s 隐式 RK4：烘干时间同为 56.6517 h；表 10 的 45 个数值全部一致，每 60 s 结果表 71421 格中仅 25 格差 1 个末位

[†] 在生产求解器上做出，是问题三结束时刻的空间收敛性依据；[‡] 为固定步长隐式 RK4 路径的专项检验，只用于说明时间离散阶与步长上限，不代表生产解的时间离散误差。含水率单位为 kg/kg。

表 15: 问题三求解过程的六项数值检验

两项收敛检验的性质需加以区分。空间网格收敛直接在生产求解器上做出，三级网格的结束时刻单调收敛，观测阶 1.562，Richardson 外推为 56.863549 h；生产结果 56.6517 h 与该外推值相差 0.212 h，小于三级网格之间的最大偏差 (0.987 h)，处于同一离散误差量级之内。时间步长收敛则是固定步长隐式 RK4 路径的专项检验，只用于确认时间离散为三阶并据此确定时间步长上限 ($\Delta t = 2.5$ s)，不代表生产解的时间离散误差——生产解的步长由自适应 BDF 的局部误差控制 ($\text{rtol} = 2 \times 10^{-7}$)。

其余四项检验分别确认：守恒格式下内部面通量严格抵消，只保留表面通量；均匀场在 24 h 内漂移恒为零，说明格式不引入虚假源项；表面含水率全程无回升，说明问题二物性过渡阶段的局部回升现象在本问中并不出现（本问物性不随阶段切换）；求解器对照则表明换用时间推进算法不改变交付口径。综合来看，问题三的结果在空间离散、时间离散、守恒性与求解器选择四个方面均得到支持。

A.3 支撑材料目录

文件名	功能说明
论文.pdf	论文正文
solve_problem1.py	问题一模型求解代码
solve_problem2.py	问题二模型求解代码
solve_problem3.py	问题三模型求解代码
solve_problem4.py	问题四模型求解代码
result_1.xlsx	
result_2.xlsx	
result_3.xlsx	
result_4.xlsx	

表 16: 支撑材料目录

A.4 核心代码

Listing 1: solve_problem1.py

```
1 """
2 问题1：预热平衡阶段 圆柱形药材的瞬态传热-传质耦合模型
3 =====
4 几何：无限长圆柱（轴向忽略），半径 R=0.02 m，一维径向 r [0,R]
5
6 控制方程（柱坐标）：
7   传热：rho*cp * dT/dt = k * (1/r) d/dr( r dT/dr )
8   传质：dC/dt          = (1/r) d/dr( r*D(C) dC/dr )
9
10 初始条件：T(r,0)=28 °C, C(r,0)=2.55 kg/kg
11 边界条件：
12   中心 r=0: 对称 dT/dr=0, dC/dr=0
13   表面 r=R: -k dT/dr = h (T - T_air(t))
14              -D dC/dr = hm (C - C_air(t))
15 其中 T_air(t), C_air(t) 为附件1的烘房温度/含湿量。
16
17 数值：有限体积法(守恒形式) + theta 隐式格式
18   温度：Crank-Nicolson (theta=0.5, 线性)
19   水分：Crank-Nicolson (theta=0.5, 非线性 D(C) 用 Picard 迭代)
20   起步：t<5s 用几何步长 dt=clip(0.05*t,0.0025,dt)，消除 t=0
        表面边界层误差
```

```

21 """
22 import numpy as np
23 import openpyxl
24 import os, sys
25 from scipy.signal import savgol_filter
26 from scipy.linalg import solve_banded
27 from pathlib import Path
28 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
29 from drying_common import (face_mean, NONLINEAR_RTOL,
30                             NONLINEAR_MAXITER, ATOL_C,
31                             ensure_deliverable_writable,
32                             save_deliverable)
33
34 try:
35     sys.stdout.reconfigure(encoding='utf-8')
36 except Exception:
37     pass
38
39 # Savitzky-Golay 保形滤波参数（与论文一致：窗宽 31 点 = 31
40 # min, 三阶多项式）
41 SMOOTH_WIN, SMOOTH_ORDER = 31, 3
42
43 # 起步几何步长：初始含水率与表面对流边界条件在  $t=0$  处不相容，
44 # 表面浓度按  $\sqrt{t}$  变化，均匀大步长会把该边界层误差放大到  $1e-3$  量级。
45 # 取  $dt_k = \text{clip}(g * t_k, dt_{min}, dt_{max})$ ，只在前几秒加密，代价可忽略。
46 EARLY_DT_MIN = 0.0025 # s, 起步最小步长
47 EARLY_DT_GROWTH = 0.05 # 每步步长按已推进时间的 5% 增长
48
49 BASE = os.path.dirname(os.path.abspath(__file__)) #
50     .../CUMCM/Problem I
51
52 PROJECT = os.path.dirname(BASE) # .../CUMCM
53 ATTACH1 = os.path.join(PROJECT, "附件", "附件1.xlsx")
54 OUT_XLSX = os.path.join(PROJECT, "result1.xlsx")
55
56 # ----- 物理参数（附录2） -----
57 R = 0.02 # 半径 m
58 rho = 820.0 # 密度 kg/m^3
59 cp = 2600.0 # 比热容 J/(kg·K)
60 k = 0.36 # 热传导系数 W/(m·K)
61 h = 25.0 # 对流换热系数 W/(m^2·K)
62 hm = 8.0e-7 # 对流传质系数 m/s
63 T0 = 28.0 # 初始温度 °C
64 C0 = 2.55 # 初始水分浓度(干基) kg/kg

```

```

60
61
62 def D_coeff(C):
63     """水分浓度扩散系数 D(C) m2/s"""
64     return 7.0e-9 * np.exp(-0.89 / C)
65
66
67 # ----- 读附件1: 烘房温度/含湿量 -----
68 def load_attachment1(path, smooth=True):
69     """读附件1 的烘房温度/含湿量; smooth=True 时先做 Savitzky-Golay
    保形滤波。"""
70     wb = openpyxl.load_workbook(path, data_only=True)
71     ws = wb.worksheets[0]
72     rows = list(ws.iter_rows(values_only=True))[1:]
73     t = np.array([r[0] for r in rows], dtype=float)
74     Tair = np.array([r[1] for r in rows], dtype=float)
75     Cair = np.array([r[2] for r in rows], dtype=float)
76     wb.close()
77     if smooth:
78         Tair = savgol_filter(Tair, SMOOTH_WIN, SMOOTH_ORDER)
79         Cair = savgol_filter(Cair, SMOOTH_WIN, SMOOTH_ORDER)
80     return t, Tair, Cair
81
82
83 class DryingSim:
84     """一维径向瞬态传热-传质求解器 (有限体积 + theta 隐式) """
85
86     def __init__(self, R, rho, cp, k, h, hm, T0, C0, Dfunc, N=400):
87         self.R, self.rho, self.cp, self.k = R, rho, cp, k
88         self.h, self.hm, self.T0, self.C0 = h, hm, T0, C0
89         self.Dfunc = Dfunc
90         self.N = N
91         self.dr = R / N
92         self.r = np.linspace(0.0, R, N + 1)
93
94         # 界面坐标 (约去 pi)
95         rp = (np.arange(N + 1) + 0.5) * self.dr
96         rm = (np.arange(N + 1) - 0.5) * self.dr
97         # 控制体体积 V'_i = r_{i+1/2}^2 - r_{i-1/2}^2
98         self.V = np.zeros(N + 1)
99         self.V[0] = (self.dr / 2.0) ** 2
100         self.V[1:N] = rp[1:N] ** 2 - rm[1:N] ** 2
101         self.V[N] = R ** 2 - rm[N] ** 2

```

```

102     # 界面面积  $A'_{i+1/2}=2*r_{i+1/2}$ 
103     self.Ap = 2.0 * rp
104     self.Am = 2.0 * rm
105
106     def _thomas(self, a, b, c, d):
107         band=np.zeros((3,len(b)))
108         band[0,1:]=c[:-1]; band[1]=b; band[2,-1]=a[1:]
109         return solve_banded((1,1),band,d,check_finite=False)
110
111     def solve_heat(self, T_old, dt, Tair_new, Tair_old, theta=0.5):
112         """温度一步: Crank-Nicolson"""
113         N = self.N; dr = self.dr
114         m = self.rho * self.cp * self.V
115
116         a = np.zeros(N + 1); b = np.zeros(N + 1); c = np.zeros(N + 1)
117         a[1:N] = self.k * self.Am[1:N] / dr
118         c[1:N] = self.k * self.Ap[1:N] / dr
119         b[1:N] = -self.k * (self.Ap[1:N] + self.Am[1:N]) / dr
120         c[0] = self.k * self.Ap[0] / dr
121         b[0] = -self.k * self.Ap[0] / dr
122         a[N] = self.k * self.Am[N] / dr
123         b[N] = -self.k * self.Am[N] / dr - 2.0 * self.R * self.h
124
125         S_next = np.zeros(N + 1); S_next[N] = 2.0 * self.R * self.h *
Tair_new
126         S_old = np.zeros(N + 1); S_old[N] = 2.0 * self.R * self.h *
Tair_old
127
128         ATold = np.zeros(N + 1)
129         ATold[1:N] = a[1:N] * T_old[0:N - 1] + b[1:N] * T_old[1:N] +
c[1:N] * T_old[2:N + 1]
130         ATold[0] = b[0] * T_old[0] + c[0] * T_old[1]
131         ATold[N] = a[N] * T_old[N - 1] + b[N] * T_old[N]
132
133         mdt = m / dt
134         lhs_b = mdt - theta * b
135         lhs_a = -theta * a
136         lhs_c = -theta * c
137         rhs = mdt * T_old + (1.0 - theta) * ATold + theta * S_next +
(1.0 - theta) * S_old
138         return self._thomas(lhs_a, lhs_b, lhs_c, rhs)
139
140     def solve_mass(self, C_old, dt, Cair_new, Cair_old, theta=0.5):

```

```

141     """水分一步: theta 隐式(Crank-Nicolson theta=0.5) + Picard
迭代"""
142     N = self.N; dr = self.dr
143     m = self.V.copy()
144
145     S_next = np.zeros(N + 1); S_next[N] = 2.0 * self.R * self.hm *
Cair_new
146     S_old = np.zeros(N + 1); S_old[N] = 2.0 * self.R * self.hm *
Cair_old
147
148     def build(C):
149         Dp = face_mean(self.Dfunc, (C[:-1],), (C[1:],))
150         Dm = np.zeros(N + 1); Dm[1:] = Dp
151         a = np.zeros(N + 1); b = np.zeros(N + 1); c = np.zeros(N +
1)
152         a[1:N] = Dm[1:N] * self.Am[1:N] / dr
153         c[1:N] = Dp[1:N] * self.Ap[1:N] / dr
154         b[1:N] = -Dp[1:N] * self.Ap[1:N] / dr - Dm[1:N] *
self.Am[1:N] / dr
155         c[0] = Dp[0] * self.Ap[0] / dr
156         b[0] = -Dp[0] * self.Ap[0] / dr
157         a[N] = Dm[N] * self.Am[N] / dr
158         b[N] = -Dm[N] * self.Am[N] / dr - 2.0 * self.R * self.hm
159         return a, b, c
160
161     def apply(a, b, c, u):
162         w = np.zeros(N + 1)
163         w[1:N] = a[1:N] * u[0:N - 1] + b[1:N] * u[1:N] + c[1:N] *
u[2:N + 1]
164         w[0] = b[0] * u[0] + c[0] * u[1]
165         w[N] = a[N] * u[N - 1] + b[N] * u[N]
166         return w
167
168     a0, b0, c0 = build(C_old)
169     LCold = apply(a0, b0, c0, C_old)
170
171     mdt = m / dt
172     C_new = C_old.copy()
173     for _ in range(NONLINEAR_MAXITER):
174         previous = C_new.copy()
175         a, b, c = build(C_new)
176         lhs_b = mdt - theta * b
177         lhs_a = -theta * a

```

```

178         lhs_c = -theta * c
179         rhs = mdt * C_old + (1.0 - theta) * LCold + theta * S_next
+ (1.0 - theta) * S_old
180         C_new = self._thomas(lhs_a, lhs_b, lhs_c, rhs)
181         if np.min(C_new)<=0 or not np.isfinite(C_new).all():
182             raise RuntimeError("水分迭代出现非法状态")
183
scale=ATOL_C+NONLINEAR_RTOL*np.maximum(np.abs(C_new),np.abs(previous))
184         if np.max(np.abs(C_new-previous)/scale)<=1:
185             break
186         else:
187             raise RuntimeError("水分非线性迭代未达到统一容差")
188         return C_new
189
190
191 def run(N=400, dt=0.5, t_end=1800.0, Tair_interp=None,
Cair_interp=None):
192     """推进 t [0,t_end], 返回 (t_out, T_hist, C_hist)"""
193     sim = DryingSim(R, rho, cp, k, h, hm, T0, C0, D_coeff, N=N)
194     if N<2 or dt<=0 or t_end<=0 or t_end!=int(t_end):
195         raise ValueError("网格至少2、dt为正、时长为正整数秒")
196     if Tair_interp is None or Cair_interp is None:
197         t_air, T_air, C_air = load_attachment1(ATTACH1)
198         Tair_interp = lambda tt: np.interp(tt, t_air, T_air)
199         Cair_interp = lambda tt: np.interp(tt, t_air, C_air)
200     t_out=np.arange(1,int(t_end)+1)
201     T_hist=np.empty((len(t_out),N+1)); C_hist=np.empty_like(T_hist)
202     T=np.full(N+1,T0); C=np.full(N+1,C0)
203     now=0.; out_idx=0; cumulative=0.
204     initial=float(sim.V*C/sim.V.sum())
205     while now<t_end-1e-10:
206         base=min(dt,max(EARLY_DT_MIN,EARLY_DT_GROWTH*now))
207         target=min(now+base,float(t_out[out_idx]),t_end)
208         step=target-now
209         ta0,ca0=Tair_interp(now),Cair_interp(now)
210         ta1,ca1=Tair_interp(target),Cair_interp(target)
211         old_surface=C[-1]
212         T=sim.solve_heat(T,step,ta1,ta0)
213         C=sim.solve_mass(C,step,ca1,ca0)
214
cumulative+=step*sim.R*sim.hm*((ca0-old_surface)+(ca1-C[-1]))/sim.V.sum()
215         now=target
216         if abs(now-t_out[out_idx])<1e-9:

```

```

217         T_hist[out_idx]=T; C_hist[out_idx]=C; out_idx+=1
218         error=abs(float(sim.V@C/sim.V.sum())-initial-cumulative)
219         run.last_balance=dict(absolute_balance_error=error,
220             relative_balance_error=error/max(abs(cumulative),1e-12))
221         return t_out, T_hist, C_hist
222
223
224 def main():
225     ensure_deliverable_writable(OUT_XLSX)          # 交付件只写
226     result1.xlsx, 占用时提前报错
227     t_air, T_air, C_air = load_attachment1(ATTACH1)
228     Tair_interp = lambda tt: np.interp(tt, t_air, T_air)
229     Cair_interp = lambda tt: np.interp(tt, t_air, C_air)
230
231     # 空间二阶收敛: t=1s 表层值误差 N=1600 约 6.3e-5 (略超
232     5e-5), N=3200 约 1.6e-5。
233     N = 3200
234     dt = 0.25
235     t_out, T_hist, C_hist = run(N=N, dt=dt, t_end=1800.0,
236         Tair_interp=Tair_interp,
237         Cair_interp=Cair_interp)
238
239     dr = R / N
240     r_out_cm = np.arange(0.0, 2.0001, 0.1)
241     r_out_idx = (r_out_cm / 100.0 / dr).round().astype(int)
242
243     t_table = [100, 300, 600, 900, 1200, 1500, 1800]
244     r_table = [0.0, 0.5, 1.0, 1.5, 2.0]
245     r_tbl_idx = [int(x / 100.0 / dr) for x in r_table]
246
247     print("\n===== 表1 药材温度(°C) =====")
248     print("时间/s\t" + "\t".join(f"r={x}cm" for x in r_table))
249     for tt in t_table:
250         i = int(tt) - 1
251         vals = [T_hist[i, idx] for idx in r_tbl_idx]
252         print(f"{tt}\t" + "\t".join(f"{v:.4f}" for v in vals))
253
254     print("\n===== 表2 药材水分浓度(kg/kg) =====")
255     print("时间/s\t" + "\t".join(f"r={x}cm" for x in r_table))
256     for tt in t_table:
257         i = int(tt) - 1
258         vals = [C_hist[i, idx] for idx in r_tbl_idx]
259         print(f"{tt}\t" + "\t".join(f"{v:.4f}" for v in vals))

```

```

257
258     # 写 result1.xlsx (以附件3 模板为底, 保留其表头样式与列宽)
259     template = os.path.join(PROJECT, "附件", "附件3", "result1.xlsx")
260     wb = openpyxl.load_workbook(template)
261     for name in ("温度", "水分浓度"):
262         ws = wb[name]
263         for row in ws.iter_rows():
264             for cell in row:
265                 cell.value = None
266         ws.cell(1, 1, "时间\\到药材中心的距离")
267         for j, x in enumerate(r_out_cm):
268             ws.cell(1, 2 + j, round(float(x), 1))
269     ws1, ws2 = wb["温度"], wb["水分浓度"]
270     for i, tt in enumerate(t_out):
271         ws1.cell(2 + i, 1, int(tt)); ws2.cell(2 + i, 1, int(tt))
272         for j, idx in enumerate(r_out_idx):
273             c1 = ws1.cell(2 + i, 2 + j, round(float(T_hist[i, idx]),
274             4))
275             c1.number_format = "0.0000"
276             c2 = ws2.cell(2 + i, 2 + j, round(float(C_hist[i, idx]),
277             4))
278             c2.number_format = "0.0000"
279         save_deliverable(wb, OUT_XLSX)
280
281 if __name__ == "__main__":
282     main()

```

Listing 2: solve_result2.py

```

1  """问题二: 二维轴对称半长度温度、水分进程并写出 result2.xlsx。
2
3  运行: python solve_result2.py
4  依赖: numpy、scipy、openpyxl。
5  """
6  from pathlib import Path
7  import argparse
8  import os
9  import sys
10 import time
11
12 WORKSPACE_DEPS = Path(__file__).resolve().parents[1] / ".p2deps"
13 if WORKSPACE_DEPS.exists():
14     sys.path.insert(0, str(WORKSPACE_DEPS))

```

```

15
16 import numpy as np
17 import openpyxl
18 from scipy.sparse import coo_matrix, diags
19 from scipy.sparse.linalg import LinearOperator, cg
20 from scipy.signal import savgol_filter
21
22
23 ROOT = Path(__file__).resolve().parent           # .../CUMCM/Problem II
24 PROJECT = ROOT.parent                           # .../CUMCM
25 ATTACHMENTS = PROJECT / "附件"
26 sys.path.insert(0, str(PROJECT))
27 from drying_common import RTOL, ensure_deliverable_writable,
    save_deliverable
28
29 # Savitzky-Golay 保形滤波参数（与论文一致：窗宽 31 点 = 31
    min, 三阶多项式）
30 SMOOTH_WIN, SMOOTH_ORDER = 31, 3
31
32
33 def read_air_data(smooth=True):
34     """读入附件1; smooth=True 时先做 Savitzky-Golay 保形滤波。"""
35     wb = openpyxl.load_workbook(ATTACHMENTS / "附件1.xlsx",
    read_only=True, data_only=True)
36     data = np.asarray(list(wb.active.values)[1:], dtype=float)
37     wb.close()
38     if smooth:
39         data[:, 1] = savgol_filter(data[:, 1], SMOOTH_WIN,
    SMOOTH_ORDER)
40         data[:, 2] = savgol_filter(data[:, 2], SMOOTH_WIN,
    SMOOTH_ORDER)
41     return data
42
43
44 def detect_critical_time(air, tail_start=10800, window=1200,
    persistence=1800):
45     """由平台接近程度和升温斜率共同识别预热结束时刻。
46
47     尾段中位数估计平台温度，MAD估计测温波动。候选时刻之前20分钟
48     的均温须进入2倍MAD带，回归斜率须不超过“3倍MAD/窗长”，且
49     两个条件此后连续保持30分钟。该判据用于离线分段，避免单点越阈。
50     """
51     t, temperature = air[:, 0], air[:, 1]

```

```

52 sample_dt = float(np.median(np.diff(t)))
53 tail = temperature[t >= tail_start]
54 if len(tail) < 5:
55     raise ValueError("附件1尾段数据不足，无法估计稳态")
56 plateau = float(np.median(tail))
57 sigma = float(1.4826 * np.median(np.abs(tail - plateau)))
58 window_points = int(round(window / sample_dt)) + 1
59 keep_points = int(round(persistence / sample_dt)) + 1
60 mean = np.full(len(t), np.nan)
61 slope = np.full(len(t), np.nan)
62 for j in range(window_points - 1, len(t)):
63     sl = slice(j - window_points + 1, j + 1)
64     minutes = (t[sl] - t[sl][0]) / 60
65     mean[j] = temperature[sl].mean()
66     slope[j] = np.polyfit(minutes, temperature[sl], 1)[0]
67 band = 2 * sigma
68 slope_limit = 3 * sigma / (window / 60)
69 stable = (np.abs(mean - plateau) <= band) & (np.abs(slope) <=
slope_limit)
70 for j in range(window_points - 1, len(t) - keep_points + 1):
71     if np.all(stable[j : j + keep_points]):
72         return float(t[j]), {
73             "plateau": plateau,
74             "sigma": sigma,
75             "band": band,
76             "slope_limit": slope_limit,
77             "window_mean": float(mean[j]),
78             "window_slope": float(slope[j]),
79             "window": window,
80             "persistence": persistence,
81         }
82     raise RuntimeError("附件1中未识别到满足持续性要求的恒温阶段")
83
84
85 def material(T,C,drying_stage):
86     """附录2/3公式对照接口；生产计算由共用核心决定物性阶段。"""
87     if np.min(C)<=0:
88         raise ValueError("含水率必须为正")
89     core=model_core()
90     time=core.CRITICAL_TIME if drying_stage else 0.
91     rho,cp,k=core.material_properties(C,time)
92     return rho*cp,k,core.moisture_diffusivity(C,T,time)
93

```

```

94
95 def model_core():
96     """复用第三问半长度核心，避免物性、边界和面系数出现两份实现。"""
97     import importlib.util
98     name="cumcm_problem3_core"
99     if name not in sys.modules:
100         spec=importlib.util.spec_from_file_location(name,
PROJECT/"Problem III/solve_problem3.py")
101         module=importlib.util.module_from_spec(spec)
102         sys.modules[name]=module
103         spec.loader.exec_module(module)
104     return sys.modules[name]
105
106
107 def solve(nr=320, nphi=None, nz=40, dt=300., end=10800, export=True,
rtol=RTOL,
108         material_mode=None, xlsx_path=None):
109     """二维轴对称半长度；nz为半长度分段数，dt为BDF最大步长上限。
110
111     前4h环境数据每60s变化，drying_common
112     的统一规则会把最大步长压到60s，
113     因此本问默认 dt=300 与第三、四问同一套时间步策略。
114     """
115     if nphi is not None:
116         raise
117     ValueError("半长度轴对称模型不再使用nphi；请删除周向网格参数")
118     if end<=0 or end!=int(end):
119         raise ValueError("输出时长必须为正整数秒")
120     started=time.time()
121     if export:
122         ensure_deliverable_writable(PROJECT / "result2.xlsx" if
xlsx_path is None else Path(xlsx_path))
123     core=model_core()
124     with core._overrides(MATERIAL_MODE=material_mode or
core.MATERIAL_MODE):
125         result=core.simulate_bdf(n_radial=nr,n_axial=nz,rtol=rtol,
126
max_step=dt,max_duration=end,record_interval=1.,stop_at_threshold=False)
127     seconds=result["time_s"][1:].astype(int)
128     sampled_T=result["temperature_c"][1:]
129     sampled_C=result["moisture_kg_per_kg"][1:]
130     for t in range(1800,int(end)+1,1800):
131         print(f"{t/3600:.1f} h: 中心温度

```

```

130     {sampled_T[t-1,0]:.6f}, 中心含水率 {sampled_C[t-1,0]:.6f}")
131     if export:
132         write_result(seconds, sampled_T, sampled_C, excel_path)
133     print("干质量归一化守恒:", result["conservation"])
134     print(f"完成, 用时 {time.time()-started:.1f} s")
135     return seconds, sampled_T, sampled_C
136
137 def write_result(seconds, temperature, moisture, output_path=None):
138     target = PROJECT / "result2.xlsx" if output_path is None else
Path(output_path)
139     wb = openpyxl.load_workbook(ATTACHMENTS / "附件3" / "result2.xlsx")
140     for name, values in (("温度", temperature), ("水分浓度",
moisture)):
141         ws = wb[name]
142         for row in ws.iter_rows():
143             for cell in row:
144                 cell.value = None
145         ws.cell(1, 1, "时间/s; 距中心距离/cm")
146         for col, radius_cm in enumerate(np.arange(21) / 10, start=2):
147             ws.cell(1, col, float(radius_cm))
148         for row, t in enumerate(seconds, start=2):
149             ws.cell(row, 1, int(t))
150             for col, value in enumerate(values[row - 2], start=2):
151                 ws.cell(row, col, round(float(value),
4)).number_format = "0.0000"
152         ws.freeze_panes = "B2"
153     return save_deliverable(wb, target)
154
155
156 if __name__ == "__main__":
157     parser =
argparse.ArgumentParser(description="计算问题二的二维轴对称半长度温度、水分进程")
158     parser.add_argument("--nr", type=int, default=320,
help="径向分段数; 320=20×16, 交付表1mm列正好是节点")
159     parser.add_argument("--nz", type=int, default=40,
help="半长度轴向分段数")
160     parser.add_argument("--dt", type=float, default=300,
help="BDF最大时间步长上限/s (前4h按共用规范压到60s)")
161     parser.add_argument("--end", type=int, default=10800,
help="结束时间/s")
162     parser.add_argument("--rtol", type=float, default=RTOL)
163     parser.add_argument("--material-mode",

```

```

choices=["specified","staged"], default=None)
164 args = parser.parse_args()
165 if args.nr < 2 or args.nz < 2 or args.dt <= 0 or args.rtol <= 0:
166     parser.error("网格数过小或时间步长不合法")
167 solve(nr=args.nr, nz=args.nz, dt=args.dt, end=args.end,
168       rtol=args.rtol, material_mode=args.material_mode)

```

Listing 3: solve_problem3.py

```

1 """第三问：药材烘房二维轴对称长时干燥模型 —— 唯一入口脚本。
2
3 本文件由原
4     q2_2d_model.py（物理库）、q3_2d_model.py（主模型）、solve_result3.py（入口）
5 与 6 个 q3_*.py 检验脚本合并而成。
6 物性默认由 drying_common.MATERIAL_MODE 选择；staged
7     为6780s附录2切换附录3，
8 specified 为题面指定的全程附录3。坐标 z 从中截面0到端面0.125m。
9
10 求解与六项检验的关键数值（烘干时间、最终最大含水率、每 6 h
11     中截面含水率表、网格与时间
12 收敛阶、水分守恒误差、均匀稳态漂移、表面回升量、RK4 与 BDF
13     对照）一律打印到终端
14
15 时间推进：BDF（容差见 drying_common.py，与问题四同一套）为生产解；RK4
16     为可复选路径。两条路径
17 终端输出会打印实际步数与右端求值次数；两种求解器的一致性由 compare
18     检验。
19
20 """
21 from __future__ import annotations
22
23 import argparse
24 import sys
25 import math
26 import time
27 from contextlib import contextmanager
28 from functools import lru_cache
29 from pathlib import Path
30
31 import numpy as np
32 from openpyxl import load_workbook
33 from scipy.integrate import solve_ivp
34 from scipy.signal import savgol_filter
35 from scipy.sparse import bmat, coo_matrix

```

```

30 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
31 from drying_common import (face_mean, component_atol, mass_balance,
    RTOL,
32     ATOL_T, ATOL_C, NONLINEAR_RTOL, NONLINEAR_MAXITER, MATERIAL_MODE,
33     ENV_FINE_INTERVAL_S, ENV_FINE_UNTIL_S,
    ensure_deliverable_writable, save_deliverable)
34
35 BASE_DIR = Path(__file__).resolve().parent
36
37
38 PROJECT = BASE_DIR.parent
39
40
41 RADIUS = 0.02
42
43
44 LENGTH = 0.25
45 HALF_LENGTH = LENGTH / 2
46
47
48 TEMPERATURE_0 = 28.0
49
50
51 MOISTURE_0 = 2.55
52
53
54 HEAT_TRANSFER_COEFFICIENT = 25.0
55
56
57 MASS_TRANSFER_COEFFICIENT = 8.0e-7
58
59
60 PLATEAU_START_TIME = 10800.0
61
62
63 SMOOTH_WIN, SMOOTH_ORDER = 31, 3
64
65
66 PHASE_BLEND = False
67
68
69 CRITICAL_WINDOW = 1200.0
70

```

```

71
72 CRITICAL_PERSISTENCE = 1800.0
73
74
75 PHASE_TRANSITION_DURATION = 1800.0
76
77
78 OUTPUT_RADII = np.arange(0.0, RADIUS + 0.0005, 0.001)
79
80
81 FULL_FIELD_TIMES = np.array([0, 1800, 3600, 5400, 7200, 9000, 10800])
82
83
84 TABLE_TIMES = np.array([1800, 3600, 5400, 7200, 9000, 10800])
85
86
87 def read_boundary_conditions() -> tuple[np.ndarray, np.ndarray,
    np.ndarray]:
88     path = PROJECT / "附件" / "附件1.xlsx"
89     workbook = load_workbook(path, data_only=True, read_only=True)
90     worksheet = workbook.active
91     rows = [
92         (float(row[0]), float(row[1]), float(row[2]))
93         for row in worksheet.iter_rows(min_row=2, values_only=True)
94         if row[0] is not None
95     ]
96     workbook.close()
97     values = np.asarray(rows, dtype=float)
98     return values[:, 0], values[:, 1], values[:, 2]
99
100
101 BOUNDARY_TIME, BOUNDARY_TEMPERATURE_RAW, BOUNDARY_MOISTURE_RAW = (
102     read_boundary_conditions()
103 )
104
105
106 BOUNDARY_TEMPERATURE = savgol_filter(
107     BOUNDARY_TEMPERATURE_RAW, SMOOTH_WIN, SMOOTH_ORDER
108 )
109
110
111 BOUNDARY_MOISTURE = savgol_filter(BOUNDARY_MOISTURE_RAW, SMOOTH_WIN,
    SMOOTH_ORDER)

```

```

112
113
114 def detect_critical_time() -> float:
115     tail_mask = BOUNDARY_TIME >= PLATEAU_START_TIME
116     tail_temperature = BOUNDARY_TEMPERATURE_RAW[tail_mask]
117     plateau_temperature = float(np.median(tail_temperature))
118     robust_sigma = float(
119         1.4826
120         * np.median(
121             np.abs(tail_temperature - plateau_temperature)
122         )
123     )
124     slope_limit = 3.0 * robust_sigma / 20.0
125
126 def satisfies(candidate_index: int) -> bool:
127     end_time = BOUNDARY_TIME[candidate_index]
128     mask = (
129         (BOUNDARY_TIME >= end_time - CRITICAL_WINDOW)
130         & (BOUNDARY_TIME <= end_time)
131     )
132     if np.count_nonzero(mask) < 5:
133         return False
134     window_time_minutes = BOUNDARY_TIME[mask] / 60.0
135     window_temperature = BOUNDARY_TEMPERATURE_RAW[mask]
136     mean_temperature = float(np.mean(window_temperature))
137     slope = float(
138         np.polyfit(
139             window_time_minutes, window_temperature, 1
140         )[0]
141     )
142     return (
143         abs(mean_temperature - plateau_temperature)
144         <= 2.0 * robust_sigma
145         and abs(slope) <= slope_limit
146     )
147
148 for candidate_index, candidate_time in enumerate(BOUNDARY_TIME):
149     if candidate_time < CRITICAL_WINDOW:
150         continue
151     if not satisfies(candidate_index):
152         continue
153     persistent = True
154     persistence_end = candidate_time + CRITICAL_PERSISTENCE

```

```

155         for check_index in range(candidate_index, len(BOUNDARY_TIME)):
156             if BOUNDARY_TIME[check_index] > persistence_end:
157                 break
158             if not satisfies(check_index):
159                 persistent = False
160                 break
161         if persistent:
162             return float(candidate_time)
163         raise RuntimeError("Could not determine the phase-switch time")
164
165
166 CRITICAL_TIME = 6780.0
167
168
169 def build_phase_weight() -> np.ndarray:
170     transition_start = CRITICAL_TIME - PHASE_TRANSITION_DURATION
171     scaled = np.clip(
172         (BOUNDARY_TIME - transition_start)
173         / PHASE_TRANSITION_DURATION,
174         0.0,
175         1.0,
176     )
177     return scaled * scaled * (3.0 - 2.0 * scaled)
178
179
180 PHASE_WEIGHT = build_phase_weight()
181
182
183 AIR_END_TIME = float(BOUNDARY_TIME[-1])
184
185
186 PLATEAU_TEMPERATURE = float(
187     np.mean(BOUNDARY_TEMPERATURE[BOUNDARY_TIME >= PLATEAU_START_TIME])
188 )
189
190
191 PLATEAU_MOISTURE = float(
192     np.mean(BOUNDARY_MOISTURE[BOUNDARY_TIME >= PLATEAU_START_TIME])
193 )
194
195
196 DRYING_AIR_TEMPERATURE = PLATEAU_TEMPERATURE
197

```

```

198
199 DRYING_AIR_MOISTURE = PLATEAU_MOISTURE
200
201
202 def air_temperature(time: float) -> float:
203     if time > AIR_END_TIME:
204         return PLATEAU_TEMPERATURE
205     return raw_air_temperature(time)
206
207
208 def air_moisture(time: float) -> float:
209     if time > AIR_END_TIME:
210         return PLATEAU_MOISTURE
211     return float(np.interp(time, BOUNDARY_TIME, BOUNDARY_MOISTURE))
212
213
214 def raw_air_temperature(time: float) -> float:
215     return float(np.interp(time, BOUNDARY_TIME, BOUNDARY_TEMPERATURE))
216
217
218 def phase_weight(time: float) -> float:
219     if MATERIAL_MODE == "specified":
220         return 1.0
221     if not PHASE_BLEND:
222         return float(time >= CRITICAL_TIME)
223     return float(np.interp(time, BOUNDARY_TIME, PHASE_WEIGHT))
224
225
226 def moisture_diffusivity(
227     concentration: np.ndarray,
228     temperature_c: np.ndarray,
229     time: float,
230 ) -> np.ndarray:
231     concentration = np.maximum(concentration, 1.0e-9)
232     temperature_k = np.maximum(temperature_c + 273.15, 1.0)
233     # 合并为单次 exp:  $\exp(-0.45/C) * \exp(-3850/T)$  与  $\exp(-0.45/C - 3850/T)$ 
234     # 数学恒等, 浮点差在 1 ulp 量级 (已逐位量化, 见验证输出)。
235     appendix3 = 2.4e-3 * np.exp(
236         -0.45 / concentration - 3850.0 / temperature_k
237     )
238     appendix2 = 7.0e-9 * np.exp(-0.89 / concentration)
239     weight = phase_weight(time)

```

```

240     return (1.0 - weight) * appendix2 + weight * appendix3
241
242
243 def material_properties(
244     concentration: np.ndarray,
245     time: float,
246 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
247     ratio = concentration / (concentration + 1.0)
248     density = 650.0 + 128.0 * concentration
249     heat_capacity = 1450.0 + 2736.0 * ratio
250     conductivity = 0.21 + 0.38 * ratio
251     weight = phase_weight(time)
252     return (
253         (1.0 - weight) * 820.0 + weight * density,
254         (1.0 - weight) * 2600.0 + weight * heat_capacity,
255         (1.0 - weight) * 0.36 + weight * conductivity,
256     )
257
258
259 @lru_cache(maxsize=None)
260 def make_geometry(
261     n_radial: int, n_axial: int
262 ) -> tuple[
263     float,
264     float,
265     np.ndarray,
266     np.ndarray,
267     np.ndarray,
268     np.ndarray,
269     np.ndarray,
270 ]:
271     """Build axisymmetric control-volume areas and volumes."""
272     radii = np.linspace(0.0, RADIUS, n_radial + 1)
273     axial_positions = np.linspace(0.0, HALF_LENGTH, n_axial + 1)
274     dr = RADIUS / n_radial
275     dz = HALF_LENGTH / n_axial
276
277     radial_left = np.empty(n_radial + 1)
278     radial_right = np.empty(n_radial + 1)
279     radial_left[0] = 0.0
280     radial_right[0] = 0.5 * (radii[0] + radii[1])
281     radial_left[-1] = 0.5 * (radii[-2] + radii[-1])
282     radial_right[-1] = RADIUS

```

```

283     if n_radial > 1:
284         radial_left[1:-1] = 0.5 * (radii[:-2] + radii[1:-1])
285         radial_right[1:-1] = 0.5 * (radii[1:-1] + radii[2:])
286
287     axial_left = np.empty(n_axial + 1)
288     axial_right = np.empty(n_axial + 1)
289     axial_left[0] = 0.0
290     axial_right[0] = 0.5 * (
291         axial_positions[0] + axial_positions[1]
292     )
293     axial_left[-1] = 0.5 * (
294         axial_positions[-2] + axial_positions[-1]
295     )
296     axial_right[-1] = HALF_LENGTH
297     if n_axial > 1:
298         axial_left[1:-1] = 0.5 * (
299             axial_positions[:-2] + axial_positions[1:-1]
300         )
301         axial_right[1:-1] = 0.5 * (
302             axial_positions[1:-1] + axial_positions[2:]
303         )
304
305     radial_cell_area = 0.5 * (
306         radial_right**2 - radial_left**2
307     )
308     axial_width = axial_right - axial_left
309     volumes = radial_cell_area[:, None] * axial_width[None, :]
310
311     radial_face_position = 0.5 * (radii[:-1] + radii[1:])
312     radial_face_area = (
313         radial_face_position[:, None] * axial_width[None, :]
314     )
315     axial_face_area = np.repeat(
316         radial_cell_area[:, None], n_axial, axis=1
317     )
318     radial_boundary_area = RADIUS * axial_width
319     axial_boundary_area = radial_cell_area
320     return (
321         dr,
322         dz,
323         volumes,
324         radial_face_area,
325         axial_face_area,

```

```

326         radial_boundary_area,
327         axial_boundary_area,
328     )
329
330
331 _WORKSPACE: dict = {}
332
333
334 def workspace(n_radial: int, n_axial: int, tag: str) -> dict:
335     key = (n_radial, n_axial, tag)
336     space = _WORKSPACE.get(key)
337     if space is None:
338         nr1, nz1 = n_radial + 1, n_axial + 1
339         space = {
340             "d_face_r": np.empty((nr1 - 1, nz1)),
341             "flux_r": np.empty((nr1 - 1, nz1)),
342             "d_face_z": np.empty((nr1, nz1 - 1)),
343             "flux_z": np.empty((nr1, nz1 - 1)),
344             "surf_z": np.empty(nz1),
345             "bnd_z": np.empty(nz1),
346             "surf_r": np.empty(nr1),
347             "bnd_r": np.empty(nr1),
348             "bnd_r2": np.empty(nr1),
349             "flux_heat": np.empty((nr1, nz1)),
350             "flux_mass": np.empty((nr1, nz1)),
351             "result": np.empty((nr1, nz1)),
352         }
353         _WORKSPACE[key] = space
354     return space
355
356
357 def conservative_operator(
358     values: np.ndarray,
359     transport_property: np.ndarray,
360     boundary_coefficient: float,
361     ambient_value: float,
362     geometry: tuple,
363     out: np.ndarray | None = None,
364     faces: tuple | None = None,
365 ) -> np.ndarray:
366     """Conservative axisymmetric divergence of a diffusive flux.
367
368     out

```

非空时写入调用方缓冲；内部临时数组按网格缓存，避免每步重复分配。
面系数由共用积分规则提供；中心对称面不计入物理边界通量。

```
369     """
370     (
371         dr,
372         dz,
373         volumes,
374         radial_face_area,
375         axial_face_area,
376         radial_boundary_area,
377         axial_boundary_area,
378     ) = geometry
379     n_radial = values.shape[0] - 1
380     n_axial = values.shape[1] - 1
381     space = workspace(n_radial, n_axial, "operator")
382     result = space["result"] if out is None else out
383     result.fill(0.0)
384
385     if faces is None:
386         raise ValueError("必须显式提供积分平均面系数")
387     flux_r = radial_face_area * faces[0] * np.diff(values, axis=0) / dr
388     flux_z = axial_face_area * faces[1] * np.diff(values, axis=1) / dz
389     result[:-1] += flux_r
390     result[1:] -= flux_r
391     result[:, :-1] += flux_z
392     result[:, 1:] -= flux_z
393
394     result /= volumes
395
396     # 侧面  $r = R$ 
397     np.multiply(radial_boundary_area, boundary_coefficient,
398 out=space["surf_z"])
399     np.subtract(values[-1], ambient_value, out=space["bnd_z"])
400     space["bnd_z"] *= space["surf_z"]
401     space["bnd_z"] /= volumes[-1]
402     result[-1] -= space["bnd_z"]
403
404     #  $z=0$  为中心对称面，无通量； $z=L/2$  为物理端面。
405     np.multiply(axial_boundary_area, boundary_coefficient,
406 out=space["surf_r"])
407     np.subtract(values[:, -1], ambient_value, out=space["bnd_r2"])
408     space["bnd_r2"] *= space["surf_r"]
409     space["bnd_r2"] /= volumes[:, -1]
```

```

409     result[:, -1] -= space["bnd_r2"]
410     return result
411
412
413 def transport_faces(concentration, temperature, time, kind):
414     function = (lambda c, t: moisture_diffusivity(c, t, time)) if kind
415     == "mass" else (
416         lambda c, t: material_properties(c, time)[2])
417     return tuple(face_mean(function, (concentration[a],
418                                     temperature[a]),
419                                     (concentration[b], temperature[b])))
420     for a, b in ((np.s_[:-1, :], np.s_[1:, :]),
421                 (np.s_[:, :-1], np.s_[:, 1:]))
422
423 def boundary_inflow(time, state, n_radial, n_axial):
424     """净入流/干质量；固定域中等于几何边界通量除以半域体积。"""
425     n = (n_radial+1)*(n_axial+1)
426     c = state[n:].reshape(n_radial+1, n_axial+1)
427     geometry = make_geometry(n_radial, n_axial)
428     ca = air_moisture(time)
429     net = MASS_TRANSFER_COEFFICIENT * (np.sum(geometry[5]*(ca-c[-1]))
430                                         + np.sum(geometry[6]*(ca-c[:,
431                                         -1]))))
432     return float(net / geometry[2].sum())
433
434 def right_hand_side(
435     state: np.ndarray,
436     time: float,
437     n_radial: int,
438     n_axial: int,
439     dr: float,
440     dz: float,
441 ) -> np.ndarray:
442     node_count = (n_radial + 1) * (n_axial + 1)
443     temperature = state[:node_count].reshape(n_radial + 1, n_axial + 1)
444     concentration = state[node_count:].reshape(
445         n_radial + 1, n_axial + 1
446     )
447
448     air_temperature_value = air_temperature(time)
449     air_moisture_value = air_moisture(time)

```

```

449     density, heat_capacity, conductivity = material_properties(
450         concentration, time
451     )
452     diffusivity = moisture_diffusivity(concentration, temperature,
time)
453     geometry = make_geometry(n_radial, n_axial)
454
455     space = workspace(n_radial, n_axial, "rhs")
456     thermal_flux = conservative_operator(
457         temperature,
458         conductivity,
459         HEAT_TRANSFER_COEFFICIENT,
460         air_temperature_value,
461         geometry,
462         out=space["flux_heat"],
463         faces=transport_faces(concentration, temperature, time,
"heat"),
464     )
465     d_temperature = thermal_flux / (density * heat_capacity)
466
467     d_concentration = conservative_operator(
468         concentration,
469         diffusivity,
470         MASS_TRANSFER_COEFFICIENT,
471         air_moisture_value,
472         geometry,
473         out=space["flux_mass"],
474         faces=transport_faces(concentration, temperature, time,
"mass"),
475     )
476
477     return np.concatenate(
478         (d_temperature.ravel(), d_concentration.ravel())
479     )
480
481
482 def interpolate_to_output_radii(
483     radii: np.ndarray, profile: np.ndarray
484 ) -> np.ndarray:
485     return np.interp(OUTPUT_RADII, radii, profile)
486
487
488 PHASE_TRANSITION_START = CRITICAL_TIME - PHASE_TRANSITION_DURATION

```

```

489
490
491 PHASE_TRANSITION_END = CRITICAL_TIME
492
493
494 THRESHOLD = 0.15
495
496
497 DT = 2.5
498
499
500 RECORD_INTERVAL = 60.0
501
502
503 MAX_DURATION = 10 * 24 * 3600
504
505
506 N_RADIAL = 320 # 交付表 1mm 列对应
      xi=k/20; 320=20*16, 交付列正好落在节点上
507
508
509 N_AXIAL = 40
510
511
512 TABLE_INTERVAL = 6 * 3600
513
514
515 TABLE_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
516
517
518 SOLVER = "bdf"
519
520
521 BDF_RTOL = RTOL
522
523
524 BDF_MAX_STEP = 300.0
525
526
527 IMPLICIT_TOLERANCE = 1.0e-7
528
529
530 IMPLICIT_MAX_ITERATIONS = NONLINEAR_MAXITER

```

```

531
532
533 IMPLICIT_RELAXATION = 0.8
534
535
536 SQRT_THREE = np.sqrt(3.0)
537
538
539 RK4_C = np.array(
540     [0.5 - SQRT_THREE / 6.0, 0.5 + SQRT_THREE / 6.0]
541 )
542
543
544 RK4_A = np.array(
545     [
546         [0.25, 0.25 - SQRT_THREE / 6.0],
547         [0.25 + SQRT_THREE / 6.0, 0.25],
548     ]
549 )
550
551
552 RK4_B = np.array([0.5, 0.5])
553
554
555 def make_state(n_radial: int, n_axial: int) -> np.ndarray:
556     node_count = (n_radial + 1) * (n_axial + 1)
557     return np.concatenate(
558         (
559             np.full(node_count, TEMPERATURE_0),
560             np.full(node_count, MOISTURE_0),
561         )
562     )
563
564
565 def profiles_from_state(
566     state: np.ndarray,
567     output_radii: np.ndarray,
568     radii: np.ndarray,
569     axial_positions: np.ndarray,
570     n_radial: int,
571     n_axial: int,
572 ) -> tuple[np.ndarray, np.ndarray]:
573     node_count = (n_radial + 1) * (n_axial + 1)

```

```

574     temperature = state[:node_count].reshape(n_radial + 1, n_axial + 1)
575     concentration = state[node_count:].reshape(
576         n_radial + 1, n_axial + 1
577     )
578     # 半长度坐标以中截面为原点，任意奇偶网格均精确包含  $z=0$ 。
579     mid_index = 0
580     midplane_temperature = temperature[:, mid_index]
581     midplane_concentration = concentration[:, mid_index]
582     return (
583         np.interp(output_radial, radial, midplane_temperature),
584         np.interp(output_radial, radial, midplane_concentration),
585     )
586
587
588 def implicit_rk4_step(
589     state: np.ndarray,
590     time: float,
591     dr: float,
592     dz: float,
593     n_radial: int | None = None,
594     n_axial: int | None = None,
595     dt: float | None = None,
596 ) -> tuple[np.ndarray, int]:
597     """Advance one step with the two-stage Gauss-Legendre implicit RK4.
598
599     缺省时取模块常量（故 `--mode temporal` 这类按需覆盖 DT
600     的检验仍然有效）；
601     局部同名变量遮蔽模块常量，函数体无需改动。
602     """
603     _globals = globals()
604     N_RADIAL = _globals["N_RADIAL"] if n_radial is None else n_radial
605     N_AXIAL = _globals["N_AXIAL"] if n_axial is None else n_axial
606     DT = _globals["DT"] if dt is None else dt
607     if not PHASE_BLEND and time < CRITICAL_TIME < time + DT:
608         middle, iterations1 = implicit_rk4_step(
609             state, time, dr, dz, N_RADIAL, N_AXIAL, CRITICAL_TIME -
610             time
611         )
612         final, iterations2 = implicit_rk4_step(
613             middle, CRITICAL_TIME, dr, dz, N_RADIAL, N_AXIAL,
614             time + DT - CRITICAL_TIME,
615         )
616     return final, max(iterations1, iterations2)

```

```

615     stage_1 = state.copy()
616     stage_2 = state.copy()
617
618     for iteration in range(1, IMPLICIT_MAX_ITERATIONS + 1):
619         derivative_1 = right_hand_side(
620             stage_1,
621             time + RK4_C[0] * DT,
622             N_RADIAL,
623             N_AXIAL,
624             dr,
625             dz,
626         )
627         derivative_2 = right_hand_side(
628             stage_2,
629             time + RK4_C[1] * DT,
630             N_RADIAL,
631             N_AXIAL,
632             dr,
633             dz,
634         )
635         next_stage_1 = state + DT * (
636             RK4_A[0, 0] * derivative_1
637             + RK4_A[0, 1] * derivative_2
638         )
639         next_stage_2 = state + DT * (
640             RK4_A[1, 0] * derivative_1
641             + RK4_A[1, 1] * derivative_2
642         )
643         scale = component_atol(len(state)//2) +
NONLINEAR_RTOL*np.maximum(
644             np.abs(next_stage_1), np.abs(next_stage_2))
645         error = max(float(np.max(np.abs(next_stage_1-stage_1)/scale)),
646                     float(np.max(np.abs(next_stage_2-stage_2)/scale)))
647         stage_1 = (
648             (1.0 - IMPLICIT_RELAXATION) * stage_1
649             + IMPLICIT_RELAXATION * next_stage_1
650         )
651         stage_2 = (
652             (1.0 - IMPLICIT_RELAXATION) * stage_2
653             + IMPLICIT_RELAXATION * next_stage_2
654         )
655         if error <= 1.0:
656             break

```

```

657     else:
658         raise RuntimeError(
659             "Implicit RK4 fixed-point iteration did not converge"
660         )
661
662     derivative_1 = right_hand_side(
663         stage_1,
664         time + RK4_C[0] * DT,
665         N_RADIAL,
666         N_AXIAL,
667         dr,
668         dz,
669     )
670     derivative_2 = right_hand_side(
671         stage_2,
672         time + RK4_C[1] * DT,
673         N_RADIAL,
674         N_AXIAL,
675         dr,
676         dz,
677     )
678     next_state = state + DT * (
679         RK4_B[0] * derivative_1 + RK4_B[1] * derivative_2
680     )
681     return next_state, iteration
682
683
684 def sparsity_pattern(n_radial: int, n_axial: int):
685
686     """状态向量的雅可比稀疏结构：同一场按网格连通，两场互相耦合（T C）。"""
687     nr1, nz1 = n_radial + 1, n_axial + 1
688     ids = np.arange(nr1 * nz1).reshape(nr1, nz1)
689     i = np.r_[ids[:-1].ravel(), ids[:, :-1].ravel()]
690     j = np.r_[ids[1:].ravel(), ids[:, 1:].ravel()]
691     rows = np.r_[ids.ravel(), i, j]
692     cols = np.r_[ids.ravel(), j, i]
693     pattern = coo_matrix(
694         (np.ones(rows.size), (rows, cols)), shape=(nr1 * nz1, nr1 *
695         nz1)
696     ).tocsr()
697     return bmat([[pattern, pattern], [pattern, pattern]], format="csr")

```

```

698 def simulate_bdf(
699     n_radial=None, n_axial=None, rtol=BDF_RTOL, max_step=BDF_MAX_STEP,
700     threshold=None, max_duration=None, record_interval=None,
701     stop_at_threshold=True, check_mass=True,
702 ):
703
704     """第二、三问共用半长度BDF核心；固定时长与达标停止只改变终止方式。"""
705     nr = N_RADIAL if n_radial is None else n_radial
706     nz = N_AXIAL if n_axial is None else n_axial
707     threshold = THRESHOLD if threshold is None else threshold
708     end = MAX_DURATION if max_duration is None else max_duration
709     interval = RECORD_INTERVAL if record_interval is None else
710     record_interval
711     if nr < 2 or nz < 2 or min(rtol, max_step, end, interval) <= 0:
712         raise ValueError("网格至少2，容差、步长和时长必须为正")
713     radii = np.linspace(0, RADIUS, nr+1)
714     axial_positions = np.linspace(0, HALF_LENGTH, nz+1)
715     dr, dz = RADIUS/nr, HALF_LENGTH/nz
716     n = (nr+1)*(nz+1)
717     def residual(t, y):
718         return right_hand_side(y, t, nr, nz, dr, dz)
719     def crossed(t, y):
720         return float(y[n:].max()-threshold)
721     crossed.terminal, crossed.direction = True, -1
722     options = dict(method="BDF", rtol=rtol,
723                     atol=component_atol(n)*(rtol/RTOL),
724                     jac_sparsity=sparsity_pattern(nr,nz),
725                     dense_output=True)
726     boundaries = sorted(set([0., float(end)] + [x for x in
727         ((CRITICAL_TIME, AIR_END_TIME) if not PHASE_BLEND else
728         (AIR_END_TIME,)) if 0<x<end]))
729     segments=[]
730     state=make_state(nr,nz)
731     for left,right in zip(boundaries[:-1],boundaries[1:]):
732         def rhs(t,y):
733             tt=np.nextafter(t,-np.inf) if right==CRITICAL_TIME and
734             t>=right else t
735             return residual(tt,y)
736         segment=solve_ivp(rhs,(left,right),state,
737             events=crossed if stop_at_threshold else None,
738             max_step=min(max_step,ENV_FINE_INTERVAL_S) if
739             left<ENV_FINE_UNTIL_S else max_step, **options)
740         if not segment.success:

```

```

735         raise RuntimeError(segment.message)
736     segments.append(segment)
737     state=segment.y[:,-1]
738     if stop_at_threshold and segment.t_events[0].size:
739         break
740     crossing=None
741     stop=float(end)
742     if stop_at_threshold:
743         if not segments[-1].t_events[0].size:
744             raise
745             RuntimeError("在允许时长内未达标，不能导出烘干完成结果")
746             crossing=float(segments[-1].t_events[0][0])
747             stop=float(math.floor(crossing)+1)
748             if stop>end:
749                 raise RuntimeError("严格达标整秒超出允许时长")
750
751     tail=solve_ivp(residual,(crossing,stop),state,max_step=max_step,**options)
752     if not tail.success or tail.y[n:,-1].max()>=threshold:
753         raise RuntimeError("严格达标整秒验证失败")
754     segments.append(tail)
755     def state_at(t):
756         for seg in segments:
757             if t<=seg.t[-1]+1e-9:
758                 return seg.sol(t)
759             raise ValueError("时间超出积分区间")
760     times=np.r_[np.arange(0.,stop,interval),stop]
761     tp,cp=[],[]
762     for t in times:
763
764         a,b=profiles_from_state(state_at(t),OUTPUT_RADII,radii,axial_positions,nr,nz)
765         tp.append(a);cp.append(b)
766         final=state_at(stop)
767         if not np.isfinite(final).all() or final[n:].min() < -ATOL_C:
768             raise RuntimeError("终态包含非法数值或负含水率")
769         balance=mass_balance(segments,n,make_geometry(nr,nz)[2],
770                               lambda t,y:boundary_inflow(t,y,nr,nz)) if
771         check_mass else None
772     return dict(time_s=times,radius_cm=OUTPUT_RADII*100,
773                xi=OUTPUT_RADII/RADIUS,
774
775     temperature_c=np.asarray(tp),moisture_kg_per_kg=np.asarray(cp),
776
777     final_time_s=stop,final_state=final,radii=radii,axial_positions=axial_positions,

```

```

772 n_radial=nr,n_axial=nz,solver="bdf",crossing_time_s=crossing,
773
774 reached_threshold=bool(stop_at_threshold),conservation=balance,
775     steps=1+sum(s.t.size-1 for s in
776 segments),nfev=sum(s.nfev for s in segments),
777     average_iterations=float("nan"))
778
779 def solve(solver: str | None = None, **kwargs) -> dict[str, object]:
780     """按 SOLVER (或显式传入的 solver) 选择时间推进。
781
782     'bdf' -> simulate_bdf (生产默认, 变阶变步长自适应); 'rk4' ->
783     simulate (可选项,
784     固定步长 + 阻尼固定点迭代)。kwargs
785     原样透传: n_radial/n_axial/threshold/
786     max_duration/record_interval 两条路径都支持, dt 仅
787     RK4、rtol/max_step 仅 BDF。
788     """
789     name = (SOLVER if solver is None else solver).lower()
790     if name == "bdf":
791         return simulate_bdf(**kwargs)
792     if name == "rk4":
793         return simulate(**kwargs)
794     raise ValueError(f"未知求解器: {name!r} (可选 'bdf' 或 'rk4') ")
795
796 def simulate(
797     n_radial: int | None = None,
798     n_axial: int | None = None,
799     dt: float | None = None,
800     threshold: float | None = None,
801     max_duration: float | None = None,
802     record_interval: float | None = None,
803 ) -> dict[str, object]:
804     """二阶段 Gauss-Legendre 隐式 RK4 推进 (可选路径)。
805
806     参数缺省时等于模块常量。函数体内以下同名的局部名遮蔽模块常量, 因此函数体
807     可按参数选择网格与步长, 输出中截面剖面。
808     """
809     _globals = globals()
810     N_RADIAL = _globals["N_RADIAL"] if n_radial is None else n_radial

```

```

808     N_AXIAL = _globals["N_AXIAL"] if n_axial is None else n_axial
809     DT = _globals["DT"] if dt is None else dt
810     THRESHOLD = _globals["THRESHOLD"] if threshold is None else
threshold
811     MAX_DURATION = _globals["MAX_DURATION"] if max_duration is None
else max_duration
812     RECORD_INTERVAL = _globals["RECORD_INTERVAL"] if record_interval
is None else record_interval
813
814     radii = np.linspace(0.0, RADIUS, N_RADIAL + 1)
815     axial_positions = np.linspace(0.0, HALF_LENGTH, N_AXIAL + 1)
816     output_radii = OUTPUT_RADII
817     dr = RADIUS / N_RADIAL
818     dz = HALF_LENGTH / N_AXIAL
819
820     state = make_state(N_RADIAL, N_AXIAL)
821     recorded_times = [0.0]
822     recorded_temperature = []
823     recorded_concentration = []
824     temperature_profile, concentration_profile = profiles_from_state(
825         state,
826         output_radii,
827         radii,
828         axial_positions,
829         N_RADIAL,
830         N_AXIAL,
831     )
832     recorded_temperature.append(temperature_profile)
833     recorded_concentration.append(concentration_profile)
834
835     time = 0.0
836     previous_time = 0.0
837     previous_state = state.copy()
838     previous_max_concentration = float(state[-len(state) // 2 :].max())
839     steps_per_record = max(1, int(round(RECORD_INTERVAL / DT)))
840     if not np.isclose(steps_per_record*DT, RECORD_INTERVAL):
841         raise ValueError("RK4
dt必须整除记录间隔；任意输出间隔请使用BDF")
842     step = 0
843     final_state = state
844     final_time = 0.0
845     iteration_total = 0
846     completed_steps = 0

```

```

847     crossing_time = None
848
849     while time < MAX_DURATION:
850         previous_time = time
851         previous_state = state.copy()
852         state, iterations = implicit_rk4_step(
853             state, time, dr, dz,
854             n_radial=N_RADIAL, n_axial=N_AXIAL, dt=DT,
855         )
856         iteration_total += iterations
857         completed_steps += 1
858         time += DT
859         step += 1
860         current_max_concentration = float(state[
861             (N_RADIAL + 1) * (N_AXIAL + 1) :
862         ].max())
863
864         if step % steps_per_record == 0:
865             temperature_profile, concentration_profile = (
866                 profiles_from_state(
867                     state,
868                     output_radii,
869                     radii,
870                     axial_positions,
871                     N_RADIAL,
872                     N_AXIAL,
873                 )
874             )
875             recorded_times.append(time)
876             recorded_temperature.append(temperature_profile)
877             recorded_concentration.append(concentration_profile)
878
879         if current_max_concentration < THRESHOLD:
880             # 达标判据与问题四一致：题目要求“低于
0.15”，故取交叉时刻之后的
881             # 下一个整秒（该时刻最大含水率严格小于阈值）。
882             denominator = (
883                 previous_max_concentration - current_max_concentration
884             )
885             if abs(denominator) < 1.0e-15:
886                 fraction = 0.0
887             else:
888                 fraction = (

```

```

889         previous_max_concentration - THRESHOLD
890     ) / denominator
891     fraction = float(np.clip(fraction, 0.0, 1.0))
892     crossing_time = previous_time + fraction * DT
893     strict_stop_time = float(math.floor(crossing_time) + 1.0)
894     # 推进到 strict_stop_time
    之后，再做线性插值（保证是插值而非外推）。
895     while time < strict_stop_time:
896         previous_time = time
897         previous_state = state.copy()
898         state, iterations = implicit_rk4_step(state, time, dr,
    dz, N_RADIAL, N_AXIAL, DT)
899         iteration_total += iterations
900         completed_steps += 1
901         time += DT
902         step += 1
903         span = time - previous_time
904         alpha = (
905             0.0
906             if span <= 0.0
907             else float(
908                 np.clip(
909                     (strict_stop_time - previous_time) / span,
    0.0, 1.0
910                 )
911             )
912         )
913         final_state = (1.0 - alpha) * previous_state + alpha *
    state
914         final_time = strict_stop_time
915         temperature_profile, concentration_profile = (
916             profiles_from_state(
917                 final_state,
918                 output_radii,
919                 radii,
920                 axial_positions,
921                 N_RADIAL,
922                 N_AXIAL,
923             )
924         )
925         if final_time - recorded_times[-1] > 1.0e-9:
926             recorded_times.append(final_time)
927             recorded_temperature.append(temperature_profile)

```

```

928         recorded_concentration.append(concentration_profile)
929         break
930
931     previous_max_concentration = current_max_concentration
932     final_state, final_time = state.copy(), time
933
934     return {
935         "time_s": np.asarray(recorded_times),
936         "radius_cm": output_radii * 100.0,
937         "xi": output_radii / RADIUS,
938         "temperature_c": np.asarray(recorded_temperature),
939         "moisture_kg_per_kg": np.asarray(recorded_concentration),
940         "final_time_s": final_time,
941         "final_state": final_state,
942         "radii": radii,
943         "axial_positions": axial_positions,
944         "n_radial": N_RADIAL,
945         "n_axial": N_AXIAL,
946         "average_iterations": iteration_total / max(completed_steps,
947 1),
948         "solver": "rk4",
949         "crossing_time_s": crossing_time,
950         "steps": int(completed_steps),
951         # 每次迭代 2 次右端求值, 循环后另加 2 次 (精确计数)
952         "nfev": int(2 * iteration_total + 2 * completed_steps),
953     }
954
955 def write_result_workbook(
956     times: np.ndarray,
957     concentrations: np.ndarray,
958     output_path: "Path | None" = None,
959 ) -> Path:
960     template_path = PROJECT / "附件" / "附件3" / "result3.xlsx"
961     output_path = (
962         PROJECT / "result3.xlsx" if output_path is None else
963         Path(output_path)
964     )
965     workbook = load_workbook(template_path)
966     worksheet = workbook["Sheet1"]
967     worksheet.delete_rows(2, worksheet.max_row)
968
969     headers = ["时间\\到药材中心的距离"] + [

```

```

969         float(f"{radius:.1f}") for radius in OUTPUT_RADII * 100.0
970     ]
971     for column, value in enumerate(headers, start=1):
972         cell = worksheet.cell(row=1, column=column, value=value)
973         cell.number_format = "0.0"
974
975     for row_index, (time, profile) in enumerate(
976         zip(times, concentrations), start=2
977     ):
978         time_cell = worksheet.cell(
979             row=row_index, column=1, value=int(round(time))
980         )
981         time_cell.number_format = "0"
982         for column, value in enumerate(profile, start=2):
983             cell = worksheet.cell(
984                 row=row_index,
985                 column=column,
986                 value=float(round(value, 4)),
987             )
988             cell.number_format = "0.0000"
989
990     worksheet.freeze_panes = "B2"
991     return save_deliverable(workbook, output_path)
992
993
994 def run_solve(
995     solver: str | None = None,
996     write_xlsx: bool = True,
997     xlsx_path: "Path | None" = None,
998 ) -> None:
999     target = PROJECT / "result3.xlsx" if xlsx_path is None else
1000     Path(xlsx_path)
1001     if write_xlsx:
1002         ensure_deliverable_writable(target)
1003     result = solve(solver)
1004     if result.get("conservation") is not None:
1005         print("干质量归一化守恒:", result["conservation"])
1006     times = result["time_s"]
1007     concentrations = result["moisture_kg_per_kg"]
1008     temperatures = result["temperature_c"]
1009     output_path = (
1010         write_result_workbook(times, concentrations, xlsx_path)
1011         if write_xlsx

```

```

1011         else None
1012     )
1013
1014
1015     table_indices = [
1016         int(np.argmin(np.abs(result["radius_cm"] - radius)))
1017         for radius in TABLE_RADII_CM
1018     ]
1019     table_rows = []
1020     for table_time in np.arange(
1021         TABLE_INTERVAL, times[-1], TABLE_INTERVAL
1022     ):
1023         index = int(np.argmin(np.abs(times - table_time)))
1024         table_rows.append(
1025             (
1026                 times[index] / 3600.0,
1027                 concentrations[index, table_indices],
1028             )
1029         )
1030     table_rows.append(
1031         (
1032             times[-1] / 3600.0,
1033             concentrations[-1, table_indices],
1034         )
1035     )
1036
1037
1038     print(f"dry time: {times[-1] / 3600.0:.6f} h")
1039     print(f"final max moisture: {concentrations[-1].max():.6f}")
1040     if result.get("solver") == "bdf":
1041         print(
1042             f"solver: BDF (rtol={BDF_RTOL:g},
1043             max_step={BDF_MAX_STEP:g} s), "
1044             f"steps: {result['steps']}, nfev: {result['nfev']}"
1045         )
1046     else:
1047         print(
1048             f"solver: RK4 (dt={DT:g} s), average implicit iterations: "
1049             f"{result['average_iterations']:.3f}, nfev: "
1050             f"{result['nfev']}"
1051         )
1052     print("\n每 6 h 的中截面含水率 (kg/kg; 径向 0 / 0.5 / 1.0 / 1.5 / 2.0 cm): ")

```

```

1051     for table_time_h, table_profile in table_rows:
1052         print(f"    {table_time_h:>8.4f} h " + "    ".join(f"{value:.4f}"
1053         for value in table_profile))
1054
1055     print(f"result workbook: {output_path}")
1056
1057 @contextmanager
1058 def _overrides(**values):
1059     """临时替换模块级全局（均匀稳态检验用），退出时逐字还原。
1060
1061     原实现直接给模型模块的属性赋值再手工还原；合并为单文件后，用上下文管理器
1062     保证异常路径下也会还原，避免污染后续求解。
1063     """
1064     missing = object()
1065     saved = {name: globals().get(name, missing) for name in values}
1066     globals().update(values)
1067     try:
1068         yield
1069     finally:
1070         for name, original in saved.items():
1071             if original is missing:
1072                 globals().pop(name, None)
1073             else:
1074                 globals()[name] = original
1075
1076
1077 def check_compare() -> None:
1078     """RK4 与 BDF 求解器对比。"""
1079     def main() -> None:
1080         started = time.perf_counter()
1081
1082         print("跑 RK4（现行主求解器）...", flush=True)
1083         mark = time.perf_counter()
1084         rk4 = simulate()
1085         time_rk4 = time.perf_counter() - mark
1086         print("    完成 %.1f s" % time_rk4, flush=True)
1087
1088         print("跑 BDF（scipy 自适应）...", flush=True)
1089         mark = time.perf_counter()
1090         bdf = simulate_bdf()
1091         time_bdf = time.perf_counter() - mark

```

```

1092     print(" 完成 %.1f s" % time_bdf, flush=True)
1093
1094     t_r = rk4["time_s"]
1095     t_b = bdf["time_s"]
1096     C_r, T_r = rk4["moisture_kg_per_kg"], rk4["temperature_c"]
1097     C_b, T_b = bdf["moisture_kg_per_kg"], bdf["temperature_c"]
1098
1099     n = int(min(t_r.size, t_b.size))
1100     dT = float(np.abs(T_r[:n] - T_b[:n]).max())
1101     dC = float(np.abs(C_r[:n] - C_b[:n]).max())
1102
1103     rounded_r = np.round(C_r[:n], 4)
1104     rounded_b = np.round(C_b[:n], 4)
1105     diff_cells = int(np.count_nonzero(rounded_r != rounded_b))
1106     rounded_gap = float(np.abs(rounded_r - rounded_b).max())
1107
1108     print()
1109     print(f"烘干时长: RK4 = {rk4['final_time_s'] / 3600:.6f} h, BDF = {bdf['final_time_s'] / 3600:.6f} h")
1110     print(f"耗时: RK4 = {time_rk4:.1f} s, BDF = {time_bdf:.1f} s, 加速 {time_rk4 / time_bdf:.1f} 倍")
1111     print(f"共同 60 s 网格前 {n} 行: max|ΔT| = {dT:.3e} °C, max|ΔC| = {dC:.3e} kg/kg")
1112     print(f"四位小数交付口径: {diff_cells} / {rounded_r.size} 个单元格不同 (最大 {rounded_gap:.0e}) ")
1113     print(f"内部步数: RK4 = {rk4['steps']} 步, BDF = {bdf['steps']} 步、nfev {bdf['nfev']}")
1114     print(f"总用时 {time.perf_counter() - started:.1f} s")
1115
1116
1117     main()
1118
1119
1120 def check_grid() -> None:
1121     """网格收敛性检验 (结果打印到终端)。"""
1122     REFINEMENT_RATIO = 2.0
1123
1124
1125     GRID_LEVELS = [
1126         ("粗网格", 12, 10),
1127         ("中网格", 24, 20),
1128         ("细网格", 48, 40),
1129     ]

```

```

1130
1131
1132     def main() -> None:
1133         records = []
1134
1135         for name, n_radial, n_axial in GRID_LEVELS:
1136             start_time = time.perf_counter()
1137             result = solve(solver="bdf", n_radial=n_radial,
n_axial=n_axial)
1138             elapsed = time.perf_counter() - start_time
1139             concentrations = result["moisture_kg_per_kg"]
1140             records.append(
1141                 {
1142                     "name": name,
1143                     "n_radial": n_radial,
1144                     "n_axial": n_axial,
1145                     "nodes": (n_radial + 1) * (n_axial + 1),
1146                     "solver": str(result.get("solver", "?")),
1147                     "steps": int(result.get("steps", 0)),
1148                     "nfev": int(result.get("nfev", 0)),
1149                     "dry_hours": float(result["crossing_time_s"] /
3600.0),
1150                     "final_max": float(concentrations[-1].max()),
1151                     "elapsed": elapsed,
1152                 }
1153             )
1154
1155             coarse = records[0]["dry_hours"]
1156             medium = records[1]["dry_hours"]
1157             fine = records[2]["dry_hours"]
1158             numerator = fine - medium
1159             denominator = medium - coarse
1160             if (
1161                 abs(numerator) < 1.0e-14
1162                 or abs(denominator) < 1.0e-14
1163                 or np.sign(numerator) != np.sign(denominator)
1164             ):
1165                 order = float("nan")
1166             else:
1167                 order = float(
1168                     np.log(abs(denominator / numerator))
1169                     / np.log(REFINEMENT_RATIO)
1170                 )

```

```

1171     if not np.isfinite(order) or order <= 0.0:
1172         richardson = float("nan")
1173     else:
1174         richardson = float(
1175             fine + (fine - medium) / (REFINEMENT_RATIO**order - 1)
1176         )
1177
1178     for record in records:
1179         print(
1180             record["name"],
1181             f"grid={record['n_radial']}x{record['n_axial']}",
1182             f"dry={record['dry_hours']:.6f} h",
1183             f"steps={record['steps']}",
1184             f"elapsed={record['elapsed']:.2f} s",
1185         )
1186         print(f"observed order: {order:.6f}")
1187         print(f"Richardson dry time: {richardson:.6f} h")
1188
1189     main()
1190
1191
1192 def check_temporal() -> None:
1193     """生产BDF容差/最大步长加密检验，固定同一空间网格。"""
1194     from validate_models import compare
1195     coarse=simulate_bdf(rtol=BDF_RTOL,max_step=BDF_MAX_STEP)
1196     fine=simulate_bdf(rtol=BDF_RTOL/10,max_step=BDF_MAX_STEP/2)
1197     compare("BDF时间精度",coarse,fine)
1198
1199
1200 def check_conservation() -> None:
1201     """生产BDF全程干质量归一化水分守恒，独立积分边界通量。"""
1202     result=simulate_bdf(check_mass=True)
1203     for key,value in result["conservation"].items():
1204         print(f"{key}: {value}")
1205
1206
1207 def check_steady() -> None:
1208     """均匀稳态检验（结果打印到终端）。"""
1209     UNIFORM_TEMPERATURE = 50.0
1210
1211
1212     UNIFORM_MOISTURE = 0.05
1213

```

```

1214
1215 TEST_DURATION = 24 * 3600
1216
1217
1218 def main() -> None:
1219     radii = np.linspace(0.0, RADIUS, N_RADIAL + 1)
1220     axial_positions = np.linspace(
1221         0.0, HALF_LENGTH, N_AXIAL + 1
1222     )
1223     dr = RADIUS / N_RADIAL
1224     dz = HALF_LENGTH / N_AXIAL
1225     node_count = (N_RADIAL + 1) * (N_AXIAL + 1)
1226
1227     state = np.concatenate(
1228         (
1229             np.full(node_count, UNIFORM_TEMPERATURE),
1230             np.full(node_count, UNIFORM_MOISTURE),
1231         )
1232     )
1233     with _overrides(
1234         air_temperature=lambda _: UNIFORM_TEMPERATURE,
1235         air_moisture=lambda _: UNIFORM_MOISTURE,
1236     ):
1237         initial_rhs = right_hand_side(
1238             state, 0.0, N_RADIAL, N_AXIAL, dr, dz
1239         )
1240         initial_max_rhs = float(np.max(np.abs(initial_rhs)))
1241
1242         step_count = int(round(TEST_DURATION / DT))
1243         for _ in range(step_count):
1244             state, _ = implicit_rk4_step(
1245                 state, 0.0, dr, dz
1246             )
1247
1248     temperature = state[:node_count].reshape(
1249         N_RADIAL + 1, N_AXIAL + 1
1250     )
1251     concentration = state[node_count:].reshape(
1252         N_RADIAL + 1, N_AXIAL + 1
1253     )
1254     maximum_temperature_drift = float(
1255         np.max(np.abs(temperature - UNIFORM_TEMPERATURE))
1256     )

```

```

1257     maximum_moisture_drift = float(
1258         np.max(np.abs(concentration - UNIFORM_MOISTURE))
1259     )
1260
1261
1262     print(f"initial RHS max: {initial_max_rhs:.6e}")
1263     print(
1264         f"24 h temperature drift: {maximum_temperature_drift:.6e}
°C"
1265     )
1266     print(
1267         f"24 h moisture drift: {maximum_moisture_drift:.6e} kg/kg"
1268     )
1269
1270     main()
1271
1272
1273 def check_rebound() -> None:
1274     """表面含水率回升检验"""
1275     TEST_DURATION = 2 * 3600
1276
1277
1278     GRID_LEVELS = [
1279         ("粗网格", 10, 20),
1280         ("中网格", 15, 30),
1281         ("细网格", 20, 40),
1282     ]
1283
1284
1285 def run_case(
1286     name: str, n_radial: int, n_axial: int
1287 ) -> dict[str, object]:
1288     result = simulate(
1289         n_radial=n_radial,
1290         n_axial=n_axial,
1291         dt=2.5,
1292         max_duration=TEST_DURATION,
1293         threshold=-1.0,
1294     )
1295
1296     times = result["time_s"]
1297     surface = result["moisture_kg_per_kg"][:, -1]
1298     differences = np.diff(surface)

```

```

1299     maximum_increase_index = int(np.argmax(differences))
1300     return {
1301         "name": name,
1302         "n_radial": n_radial,
1303         "n_axial": n_axial,
1304         "maximum_rebound":
1305         float(differences[maximum_increase_index]),
1306         "rebound_time": float(times[maximum_increase_index]),
1307         "surface_at_rebound": float(
1308             surface[maximum_increase_index]
1309         ),
1310         "surface_final": float(surface[-1]),
1311     }
1312
1313 def main() -> None:
1314     results = [
1315         run_case(name, n_radial, n_axial)
1316         for name, n_radial, n_axial in GRID_LEVELS
1317     ]
1318
1319     for result in results:
1320         print(
1321             result["name"],
1322             f"rebound={result['maximum_rebound']:.6f}",
1323             f"time={result['rebound_time']:.1f} s",
1324         )
1325
1326     main()
1327
1328
1329
1330 MODES = {
1331     "compare": ("RK4 与 BDF 求解器对比", check_compare),
1332     "grid": ("网格收敛性检验", check_grid),
1333     "temporal": ("时间积分精度检验 (生产BDF)", check_temporal),
1334     "conservation": ("全局水分守恒检验", check_conservation),
1335     "steady": ("均匀稳态检验", check_steady),
1336     "rebound": ("表面含水率回升检验", check_rebound),
1337 }
1338
1339
1340 def main() -> int:

```

```

1341 global MATERIAL_MODE, N_RADIAL, N_AXIAL
1342 parser = argparse.ArgumentParser(
1343     prog="solve_problem3.py",
1344     description="第三问：二维轴对称长时干燥模型（求解 +
    检验，唯一入口）。",
1345     formatter_class=argparse.RawDescriptionHelpFormatter,
1346     epilog=__doc__,
1347 )
1348 parser.add_argument(
1349     "--mode",
1350     default="solve",
1351     choices=["solve", *MODES, "all"],
1352     help="solve=求解并写交付件（默认）；all=求解 + 全部检验",
1353 )
1354 parser.add_argument(
1355     "--solver",
1356     choices=["bdf", "rk4"],
1357     default=None,
1358     help="时间推进方式，默认取模块常量 SOLVER（bdf）",
1359 )
1360 parser.add_argument("--material-mode", choices=["specified",
    "staged"], default=MATERIAL_MODE)
1361 parser.add_argument("--nr", type=int, default=N_RADIAL)
1362 parser.add_argument("--nz", type=int, default=N_AXIAL,
    help="半长度分段数")
1363 args = parser.parse_args()
1364 if args.nr < 2 or args.nz < 2:
1365     parser.error("网格分段数至少2")
1366 MATERIAL_MODE, N_RADIAL, N_AXIAL = args.material_mode, args.nr,
    args.nz
1367 if MATERIAL_MODE == "staged":
1368     print("物性：6780
    s前附录2，之后附录3（额外分段假设；题面统一附录用 --material-mode
    specified）")
1369
1370 if args.mode in ("solve", "all"):
1371     run_solve(args.solver)
1372
1373 if args.mode == "all":
1374     for label, function in MODES.values():
1375         print(f"\n{'=' * 30} {label} {'=' * 30}")
1376         function()
1377 elif args.mode != "solve":

```

```

1378         MODES[args.mode][1]()
1379         return 0
1380
1381
1382 if __name__ == "__main__":
1383     raise SystemExit(main())

```

Listing 4: solve_problem4.py

```

1  """问题四：二维轴对称半长度收缩模型（数值规范共用 drying_common.py）。
2
3  敏感性/对照/网格检查跑法只算校验量、不落盘（validate_models.py 用
    --validation_only 取内存结果）：
4      python solve_problem4.py --no-shrink                # 不收缩对照
5      python solve_problem4.py --material-mode specified  #
    全程附录4对照
6  坐标：xi=r/R(t)，z 为中截面到端面的距离，长度保持 25 cm。
7  干基含水率跟随固体材料运动，采用均匀径向收缩，不加入体积浓度稀释项。
8  环境边界条件（附件1）先做 Savitzky-Golay
    保形滤波（窗宽31点、三阶），与问题一、二一致。
9  """
10 from pathlib import Path
11 import argparse
12 import sys
13 import json
14 import os
15 import time
16 import numpy as np
17 from scipy.integrate import solve_ivp
18 from scipy.interpolate import PchipInterpolator
19 from scipy.sparse import coo_matrix, bmat
20 from scipy.signal import savgol_filter
21 from openpyxl import load_workbook
22 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
23 from drying_common import (face_mean, component_atol, mass_balance,
    RTOL, MATERIAL_MODE,
24
25                             ENV_FINE_INTERVAL_S, ENV_FINE_UNTIL_S,
    ensure_deliverable_writable,
    save_deliverable)
26
27 # 环境边界条件的 Savitzky-Golay 保形滤波参数，与问题一、二同一套
28 SMOOTH_WIN, SMOOTH_ORDER = 31, 3
29
30

```

```

31 def read_attachment(path):
32     wb = load_workbook(path, read_only=True, data_only=True)
33     a = np.array(list(wb.active.values)[1:], dtype=float)
34     wb.close()
35     assert np.isfinite(a).all() and np.all(np.diff(a[:, 0]) > 0)
36     return a
37
38
39 class DryingModel:
40     def __init__(self, project, nr=320, nz=40, shrink=True,
41 preheat=False, smooth=True, material_mode=MATERIAL_MODE):
42         self.air = read_attachment(project / '附件/附件1.xlsx')
43         if smooth:
44             # 环境边界条件先做 Savitzky-Golay 保形滤波 (窗宽31点即31
45 min、三阶多项式)
46             self.air[:, 1] = savgol_filter(self.air[:, 1], SMOOTH_WIN,
47 SMOOTH_ORDER)
48             self.air[:, 2] = savgol_filter(self.air[:, 2], SMOOTH_WIN,
49 SMOOTH_ORDER)
50         self.radius_data = read_attachment(project / '附件/附件2.xlsx')
51         assert np.all(np.diff(self.radius_data[:, 1]) <= 0)
52         self.rfit = PchipInterpolator(self.radius_data[:, 0],
53 self.radius_data[:, 1] / 100)
54         self.shrink, self.preheat, self.smooth = shrink, preheat,
55 smooth
56         self.material_mode = "smooth" if preheat else material_mode
57         tail = self.air[self.air[:, 0] >= 10800, 1:]
58         self.plateau = np.mean(tail, axis=0)
59         self.nr, self.nz = nr, nz
60         self.x = np.linspace(0, 1, nr + 1)
61         self.z = np.linspace(0, .125, nz + 1)
62         self.dx, self.dz = 1 / nr, .125 / nz
63         faces = np.r_[0, (self.x[:-1] + self.x[1:]) / 2, 1]
64         self.rweight = np.diff(faces**2) / 2
65         self.zweight = np.full(nz + 1, self.dz)
66         self.zweight[[0, -1]] /= 2
67         self.shape = (nr + 1, nz + 1)
68         self.n = (nr + 1) * (nz + 1)
69         ids = np.arange(self.n).reshape(self.shape)
70         i = np.r_[ids[:-1].ravel(), ids[:, :-1].ravel()]
71         j = np.r_[ids[1:].ravel(), ids[:, 1:].ravel()]
72         rows, cols = np.r_[ids.ravel(), i, j], np.r_[ids.ravel(), j, i]
73         S = coo_matrix((np.ones(len(rows)), (rows, cols)),

```

```

shape=(self.n, self.n)).tocsr()
68     self.sparsity = bmat([[S, S], [S, S]], format='csr')
69
70     def radius(self, t):
71         if not self.shrink:
72             return .02
73         # 附件覆盖到72h, 主计算不允许依靠无依据的半径外推。
74         return float(self.rfit(np.clip(t, 0, self.radius_data[-1, 0])))
75
76     def ambient(self, t):
77         if t > self.air[-1, 0]:
78             return self.plateau
79         return np.array([np.interp(t, self.air[:, 0], self.air[:, j])
80 for j in (1, 2)])
81
82     def phase_weight(self, t):
83         if self.material_mode == "specified":
84             return 1.
85         if self.material_mode == "staged":
86             return float(t >= 6780.)
87         x = np.clip((t-4980.)/1800.,0.,1.)
88         return x*x*(3-2*x)
89
90     def material(self, t, T, C):
91         c = np.maximum(C, 1e-9) #
92         仅保护牛顿迭代的试探值; 最终场单独验证。
93         rho = 760 + 90 * c
94         cp = 1850 + 2150 * c / (1 + c)
95         k = .12 + .20 * c / (1 + c)
96         D = 4.2e-4 * np.exp(-.30 / c - 3850 / (T + 273.15))
97         w = self.phase_weight(t)
98         rho = (1-w)*820 + w*rho
99         cp = (1-w)*2600 + w*cp
100         k = (1-w)*.36 + w*k
101         D = (1-w)*7e-9*np.exp(-.89/c) + w*D
102         return rho*cp, k, D
103
104     def transport_faces(self, t, T, C, kind="mass"):
105         index = 2 if kind == "mass" else 1
106         function = lambda c,temp: self.material(t,temp,c)[index]
107         return tuple(face_mean(function,(C[a],T[a]),(C[b],T[b]))
108 for a,b in
((np.s_[:-1,:],np.s_[1:,:]),(np.s_[::-1],np.s_[::-1])))

```

```

107
108     def moisture_faces(self, t, T, C):
109         return self.transport_faces(t, T, C, "mass")
110
111     def boundary_inflow(self, t, y):
112         C = y[self.n:].reshape(self.shape)
113         R = self.radius(t)
114         ca = self.ambient(t)[1]
115         net = 8e-7 * (np.sum((ca - C[-1]) * R * self.zweight) +
116                      np.sum((ca - C[:, -1]) * self.rweight * R ** 2))
117         volume = R ** 2 * self.rweight.sum() * self.zweight.sum()
118         return float(net / volume)
119
120     def diffusion(self, f, a, h, ambient, R, faces=None):
121         """节点控制体通量，省略各式相同的周向因子2pi。"""
122         if faces is None and not np.all(a == a.flat[0]):
123             raise ValueError("非均匀物性必须显式传入积分平均面系数")
124         radial_area = self.rweight * R ** 2
125         V = radial_area[:, None] * self.zweight[None, :]
126         q = np.zeros_like(f)
127         af = faces[0] if faces is not None else
128         np.broadcast_to(a[:-1], f[:-1].shape)
129         flux = af * (f[1:] - f[:-1]) *
130         ((self.x[:-1] + self.x[1:]) / 2 / self.dx)[:, None] * self.zweight
131         q[:-1] += flux
132         q[1:] -= flux
133         af = faces[1] if faces is not None else np.broadcast_to(a[:,
134         :-1], f[:, :-1].shape)
135         flux = af * (f[:, 1:] - f[:, :-1]) / self.dz * radial_area[:,
136         None]
137         q[:, :-1] += flux
138         q[:, 1:] -= flux
139         q[-1] += h * (ambient - f[-1]) * R * self.zweight
140         q[:, -1] += h * (ambient - f[:, -1]) * radial_area
141         return q / V
142
143     def rhs(self, t, y):
144         T, C = y[:self.n].reshape(self.shape),
145         y[self.n:].reshape(self.shape)
146         cap, k, D = self.material(t, T, C)
147         Ta, Ca = self.ambient(t)
148         R = self.radius(t)
149         return np.r_[(self.diffusion(T, k, 25, Ta, R,

```

```

145         self.transport_faces(t,T,C,"heat"))/cap).ravel(),
        self.diffusion(C, D, 8e-7, Ca, R,
146         self.moisture_faces(t,T,C)).ravel()])
147
148     def profiles(self, y):
149         C = y[self.n:].reshape(self.shape)
150         return C @ self.zweight / .125, C[:, 0], float(C.max()),
151         float(C.min())
152
153     def checks(self, t, y):
154         T, C = y[:self.n].reshape(self.shape),
155         y[self.n:].reshape(self.shape)
156         _, _, D = self.material(t, T, C)
157         R = self.radius(t)
158         Ca = self.ambient(t)[1]
159         V = self.rweight[:, None]*R**2*self.zweight
160         dC = self.diffusion(C, D, 8e-7, Ca, R,
161         self.moisture_faces(t,T,C))
162         outward = 8e-7*(np.sum((C[-1]-Ca)*R*self.zweight) +
163         np.sum((C[:, -1]-Ca)*self.rweight*R**2))
164         balance = float(abs(np.sum(V*dC)+outward)/max(abs(outward),
165         1e-30))
166         eqT, eqC = np.full(self.shape, 50.), np.full(self.shape, .05)
167         _, k, D = self.material(t, eqT, eqC)
168         eq = max(np.abs(self.diffusion(eqT,k,25,50.,R)).max(),
169         np.abs(self.diffusion(eqC,D,8e-7,.05,R)).max())
170         return {'relative_instantaneous_flux_residual':balance,
171         'uniform_equilibrium_rhs':float(eq)}
172
173     def run(args):
174         started = time.perf_counter()
175         target = xlsx_target(args)
176         if target is not None:
177             ensure_deliverable_writable(target)
178         m = DryingModel(args.project, args.nr, args.nz, not
179         args.no_shrink, args.preheat, not args.no_smooth,
180         args.material_mode)
181         y0 = np.r_[np.full(m.n,28.), np.full(m.n,2.55)]
182         def event(t, y):
183             return float(np.max(y[m.n:])) - .15
184         event.terminal, event.direction = True, -1
185         #

```

前4h环境数据每60s变化，按共用规范限制内部步长；后续仍连续继承全部状态。

```
177     segments = []
178     y = y0
179     bounds=sorted(set([0.,ENV_FINE_UNTIL_S,259200.]+([6780.] if
m.material_mode=="staged" else [])))
180     for start,end in zip(bounds[:-1],bounds[1:]):
181         step=min(args.max_step,ENV_FINE_INTERVAL_S) if
start<ENV_FINE_UNTIL_S else args.max_step
182         def rhs(t,y):
183             tt=np.nextafter(t,-np.inf) if end==6780. and t>=end else t
184             return m.rhs(tt,y)
185         sol =
solve_ivp(rhs,(start,end),y,method='BDF',rtol=args.rtol,atol=component_atol(m.n)*(
186
jac_sparsity=m.sparsity,max_step=step,events=event,dense_output=True)
187         if not sol.success:
188             raise RuntimeError(sol.message)
189         segments.append(sol)
190         y = sol.y[:,-1]
191         print(f'grid={args.nr}x{args.nz}, t={sol.t[-1]/3600:.6f} h,
Cmax={y[m.n:].max():.9f}',flush=True)
192         if sol.t_events[0].size:
193             break
194         if not segments[-1].t_events[0].size:
195             raise
RuntimeError('72小时内未达标，附件2不足以确定更晚半径，未生成伪造完成结果。')
196         crossing = float(segments[-1].t_events[0][0])
197         # 等于阈值是临界时刻；另算下一整秒以满足严格小于。
198         stop = float(np.floor(crossing)+1)
199         if stop > 259200:
200             raise RuntimeError("严格达标时刻超出半径数据覆盖时长")
201         final =
solve_ivp(m.rhs,(crossing,stop),y,method='BDF',rtol=args.rtol,atol=component_atol(
202             jac_sparsity=m.sparsity,dense_output=True)
203         assert final.success and final.y[m.n:,-1].max() < .15
204         def state(t):
205             if t > crossing:
206                 return final.sol(t)
207             for segment in segments:
208                 if t <= segment.t[-1]+1e-9:
209                     return segment.sol(t)
210             raise ValueError("时间超出积分区间")
211         summary =
```

```

212 {'nr':args.nr,'nz_half':args.nz,'rtol':args.rtol,'max_step_s':args.max_step,
213     'smooth_env':bool(m.smooth),'smooth_win':SMOOTH_WIN,'smooth_order':SMOOTH_ORDER,
214     'shrink':not
args.no_shrink,'preheat_appendix2':m.material_mode!='specified',
215     'material_mode':m.material_mode,'profile_mode':'midplane','schema_version':2,
216     'conservation':mass_balance(segments+[final],m.n,
m.rweight[:,None]*m.zweight[None,:],m.boundary_inflow),
217     'threshold_crossing_s':crossing,'threshold_crossing_h':crossing/3600,
218     'strict_stop_s':stop,'strict_stop_h':stop/3600,
219     'radius_stop_cm':m.radius(stop)*100,'max_C_stop':float(final.y[m.n:,-1].max()),
220     'air_plateau_T_C':m.plateau.tolist(),'seconds_runtime':time.perf_counter()-started
221     'max_C_one_second_before_crossing':float(state(crossing-1)[m.n:].max()),
222     'checks':m.checks(stop,final.y[:, -1]),
223     'nfev':sum(s.nfev for s in segments)}
224 if getattr(args, 'validation_only', False):
225     times=np.r_[np.arange(0.,stop,60.),stop]
226     temperatures,moistures=[],[]
227     # 与第三问共用的探针：归一化半径
228     xi=r/R(t)=0,0.05,...,1, 共21列。
229     # radius_cm 记录参考构型半径 xi*2cm; 收缩时实际半径再乘
230     R(t)/2cm。
231     xi=np.linspace(0.,1.,21)
232     for t in times:
233         vector=state(t)
234         T=vector[:,m.n:].reshape(m.shape)[:,-1]
235         C=vector[m.n:].reshape(m.shape)[:,-1]
236         temperatures.append(np.interp(xi,m.x,T))
237         moistures.append(np.interp(xi,m.x,C))
238     return dict(time_s=times,xi=xi,radius_cm=xi*2.,
239                 temperature_c=np.asarray(temperatures),
240                 moisture_kg_per_kg=np.asarray(moistures),crossing_time_s=crossing,
241                 conservation=summary['conservation'],summary=summary)
242 times = np.r_[np.arange(60,stop,60),stop]
radii_cm = np.arange(21)/10
records = []

```

```

243     for t in times:
244         avg, mid, cmax, cmin = m.profiles(state(t))
245         Rcm = m.radius(t)*100
246         # 口径: 输出中截面  $z=L/2$  的径向剖面 (此前用 avg 轴向平均)
247         vals = [float(np.interp(r/Rcm,m.x,mid)) if r <= Rcm+1e-10 else
None for r in radii_cm]
248         records.append([float(t)]+vals+[float(mid[-1])])
249         table6=[]
250         for t in np.r_[np.arange(21600,stop,21600),stop]:
251             avg, mid, cmax, cmin = m.profiles(state(t))
252             Rcm = m.radius(t)*100
253             # 口径: 表6 同用中截面; “药材表面”列 =  $mid[-1] = C(R(t),$ 
 $L/2, t)$ 
254             vals = [float(np.interp(r/Rcm,m.x,mid)) if r <= Rcm+1e-10 else
None for r in (0,.5,1,1.5)]
255             table6.append([float(t/3600)]+vals+[float(mid[-1]),Rcm,cmax])
256
257         payload={'headers':['时间/s; 距离/cm']+radii_cm.tolist()+['药材表面'],
258                 'rows':records,
259                 'table6_headers':['时间/h','0 cm','0.5 cm','1.0 cm','1.5
cm','药材表面','半径/cm','全场最大含水率'],
260                 'table6':table6}
261         target = xlsx_target(args)
262         if target is None:
263
264             print('未写交付件: 本次为敏感性/对照跑法, 只算校验量、不落盘',flush=True)
265             else:
266                 export_result4(payload, args.project, target)
267             print(json.dumps(summary,ensure_ascii=False),flush=True)
268
269 def xlsx_target(args):
270     """交付件 result4.xlsx 的写出路径; 敏感性/对照跑法返回 None。"""
271     variants = []
272     if args.no_shrink:
273         variants.append('no_shrink')
274     if getattr(args, 'preheat', False):
275         variants.append('smooth_material')
276     if args.material_mode != MATERIAL_MODE:
277         variants.append(args.material_mode)
278     if args.no_smooth:
279         variants.append('no_smooth')
280     if variants:
281         # 对照/敏感性跑法只算校验量

```

```

280         return None
281     return args.project/'result4.xlsx'
282
283
284 def export_result4(payload, project, out):
285
286     """按“附件3/result4.xlsx”模板写出交付件：四位小数、冻结首行首列、超出半径留空。”
287     headers, rows = payload['headers'], payload['rows']
288     if len(headers) != len(rows[0]):
289         raise SystemExit('表头与数据列数不符： %d vs %d' %
290 (len(headers), len(rows[0])))
291     wb = load_workbook(project/'附件3/'/'result4.xlsx')
292     ws = wb['Sheet1']
293     for row in ws.iter_rows():
294         for c in row:
295             c.value = None
296     for j, h in enumerate(headers, start=1):
297         ws.cell(1, j, h)
298     for i, rec in enumerate(rows, start=2):
299         ws.cell(i, 1, int(round(rec[0])))
300         for j, v in enumerate(rec[1:], start=2):
301             if v is None:
302                 continue # 超出当前半径：留空
303             ws.cell(i, j, round(float(v), 4)).number_format = '0.0000'
304     ws.freeze_panes = 'B2'
305     ws.column_dimensions['A'].width = 19.625
306     for col in 'BCDEFGHIJKLMNOPQRSTUVWXYZ':
307         ws.column_dimensions[col].width = 9.625
308     shape = (ws.max_row, ws.max_column)
309     saved = save_deliverable(wb, out)
310     print(' 行数=%d 列数=%d' % shape, flush=True)
311     return saved
312
313 if __name__ == '__main__':
314     p=argparse.ArgumentParser(description=__doc__)
315
316     p.add_argument('--project', type=Path, default=Path(__file__).resolve().parent.parent)
317
318     p.add_argument('--nr', type=int, default=320, help='径向单元数；取20的倍数时交付表1mm')
319
320     p.add_argument('--nz', type=int, default=40, help='半长度方向的单元数')
321     p.add_argument('--rtol', type=float, default=RTOL)

```

```

318     p.add_argument('--max-step',type=float,default=300)
319
320     p.add_argument('--no-shrink',action='store_true',help='对照：不收缩')
321
322     p.add_argument('--material-mode',choices=['specified','staged','smooth'],default=M
323
324     p.add_argument('--preheat',action='store_true',help='敏感性：使用附录2预热物性的平
325
326     p.add_argument('--no-smooth',action='store_true',help='敏感性：环境输入不做SG平滑,
327     a=p.parse_args()
328     if a.nr<4 or a.nz<4 or a.rtol<=0 or a.max_step<=0:
329         p.error('网格、容差和步长必须为正，网格至少4。')
330     run(a)

```